

Home

KristijanOresic edited this page on Oct 21, 2025 · [12 revisions](#)

Edit

New page

Dobrodošli na TravelMate GitHub Wiki gdje se nalazi projektna dokumentacija za TravelMate web aplikaciju.

U nastavku možete vidjeti sve bitne informacije o projektu.

Programsko inženjerstvo ak.god 2025./2026

Sveučilište u Zagrebu

Fakultet elektrotehnike i računarstva

TravelMate

Tim: <TG11.3>

TravelMates

Nastavnik: Vlado Sruk

TravelMates

✎

▼ Pages 13

Find a page or section...

▼ Home

Programsko inženjerstvo ak.god 2025./2026

Sveučilište u Zagrebu

Fakultet elektrotehnike i računarstva

TravelMate

Tim: <TG11.3>

TravelMates

Nastavnik: Vlado Sruk

▶ 1. Opis projektnog zadatka

▶ 2. Analiza zahtjeva

▶ 3. Specifikacija zahtjeva sustava

▶ 4. Arhitektura i dizajn sustava

▶ 5. Arhitektura komponenata i r...

▶ 6. Ispitivanje programskog rješ...

▶ 7. Tehnologije za implementaci...

▶ 8. Upute za puštanje u pogon

▶ 9. Zaključak i budući rad

▶ A. Popis literature

▶ A. Dnevnik promjena dokume...

▶ B. Prikaz aktivnosti grupe

+ Add a custom sidebar

Clone this wiki locally

<https://github.com/KristijanOr>

📄

[Edit](#) [New page](#)

KristijanOresic edited this page on Oct 21, 2025 · [13 revisions](#)

!!

U današnje vrijeme turizam se sve više oslanja na digitalne tehnologije i dostupnost informacija u stvarnom vremenu. Turisti očekuju brze, točne i personalizirane podatke o destinacijama koje posjećuju, a klasični turistički vodiči često ne mogu zadovoljiti te potrebe zbog svoje statičnosti i ograničenog opsega informacija. Osim toga, suvremeni turisti sve više preferiraju samostalno istraživanje gradova i znamenitosti, bez organiziranih tura ili vodiča. Takav način putovanja daje veću slobodu, ali otvara i niz problema – posjetitelji često nemaju pristup kvalitetnim, lokalno relevantnim informacijama, a dostupni izvori (poput nasumičnih internetskih stranica) mogu biti nepouzdana, nepotpuni ili zastarjeli. Mnogi turisti ne znaju koje lokacije uopće vrijedi posjetiti, niti poznaju njihovu povijest, značenje i zanimljivosti, pa njihovo iskustvo ostaje površno. Uz to, sve veći broj putnika dolazi iz različitih zemalja i kultura, pa je potrebno osigurati raznolik i pristupačan način prezentacije informacija, ne samo kroz tekst, već i putem multimedije – zvuka, slika i interaktivnih karata. Klasični vodiči u tom smislu ne pružaju fleksibilnost, jer su statični i ne mogu se prilagođavati potrebama korisnika.

S obzirom na sve navedeno, javlja se potreba za modernim, integriranim rješenjem koje će spojiti funkcionalnosti digitalnog turističkog vodiča, interaktivne mape i multimedijalnog sadržaja.

Projekt TravelMate nastao je kao odgovor na te izazove. Cilj projekta je razviti web platformu koja turistima i lokalnim posjetiteljima omogućuje jednostavno otkrivanje i istraživanje znamenitosti putem interaktivne karte. Korisnici mogu dobiti detaljne informacije o atrakcijama, pogledati slike, pročitati povijesne podatke, te pomoću integriranog Text-to-Speech sustava poslušati audio opis svake znamenitosti, čime se aplikacija pretvara u praktičan audio vodič. Time se korisniku omogućuje hands-free iskustvo razgledavanja, što povećava dostupnost informacija i osobnu udobnost tijekom putovanja.

S druge strane, platforma omogućuje i urednicima (editorima) da jednostavno dodaju nove atrakcije, ažuriraju postojeće podatke i održavaju sadržaj relevantnim i točnim. Time se osigurava dugoročna održivost i točnost informacija.

Glavne funkcionalnosti web aplikacije TravelMate su :

- omogućiti prikaz svih znamenitosti na interaktivnoj Google mapi
- prikazati detaljne informacije o svakoj atrakciji (povijesne činjenice, slike, radno vrijeme i dr.)
- implementirati mogućnost pretvaranja teksta u govor (Text-to-Speech) kako bi korisnici mogli slušati opise znamenitosti
- omogućiti korisnicima spremanje atrakcija u favorite

▼

Pages 13

Find a page or section...

► Home

▼

1. Opis projektnog zadatka

Detaljan opis problema, ciljeva i koristi sustava

Razrada problematike zadatka ! !

Potencijalna korist, opseg i cilj ovog projekta ✓

Ciljani skup korisnika 👤

Turisti i posjetitelji

Lokalne zajednice i turističke organizacije

Urednici i administratori

Prilagodbe rješenja 🔧

✚

Prilagodba audio funkcionalnosti

Prilagodba korisničkih uloga

Prilagodba dizajna i responzivnost aplikacije

Moguće nadogradnje zadatka 🏗️

Poboljšanje korisničkog iskustva (UI)

Funkcionalne nadogradnje

Proširenje audio funkcionalnosti

Sigurnost i autentifikacija

Analitika

Slična rješenja 🌐

Google Maps

izi.Travel

TripAdvisor

► 2. Analiza zahtjeva

► 3. Specifikacija zahtjeva sustava

► 4. Arhitektura i dizajn sustava

► 5. Arhitektura komponenta i r...

- razviti sustav autentifikacije s mogućnošću registracije, prijave i vanjske prijave putem Google računa (OAuth)
- omogućiti urednicima (editorima) dodavanje i ažuriranje sadržaja putem jednostavnog sučelja

TravelMate spaja praktičnost i jednostavnost: turisti brzo dobivaju potrebne informacije, spremaju favorite i mogu slušati opise znamenitosti, dok urednici održavaju sadržaj ažurnim i zanimljivim.

Ciljani skup korisnika

Turisti i posjetitelji

Za turiste i posjetitelje, TravelMate predstavlja praktičan i intuitivan alat koji im omogućuje jednostavno pronalaženje i istraživanje znamenitosti u destinaciji koju posjećuju. Kroz interaktivno sučelje korisnici mogu brzo doći do točnih, sažetih i zanimljivih informacija o povijesti, kulturi i zanimljivostima svake lokacije. Dodatno, mogućnost slušanja audio opisa putem integriranog Text-to-Speech sustava omogućuje im hands-free iskustvo razgledavanja, što posebno doprinosi praktičnosti tijekom kretanja gradom. Sustav favorita pruža personalizirano iskustvo, jer korisnici mogu spremati lokacije koje ih zanimaju i kasnije im se lako vraćati.

Lokalne zajednice i turističke organizacije

S aspekta lokalnih zajednica i turističkih organizacija, projekt donosi dugoročne koristi u vidu povećanja vidljivosti kulturnih i povijesnih znamenitosti. Pomoću TravelMate platforme moguće je promovirati i manje poznate lokacije koje često ostaju izvan glavnih turističkih ruta, čime se potiče ravnomjernija raspodjela posjetitelja i smanjuje prenapučenost popularnih područja. Time projekt doprinosi razvoju održivog turizma, potiče lokalnu ekonomiju te promiče kulturnu baštinu na suvremen i pristupačan način.

Urednici i administratori

Za urednike i administratore, TravelMate omogućuje jednostavno i učinkovito upravljanje sadržajem. Urednici mogu brzo dodavati nove atrakcije, ažurirati postojeće informacije te reagirati na promjene u radnom vremenu, cijenama ili događanjima. Takav pristup osigurava da sustav ostane ažuran, relevantan i pouzdan, dok istovremeno smanjuje potrebu za tehničkim znanjem pri upravljanju podacima. Time se povećava održivost projekta i omogućuje njegovo lako proširenje u budućnosti.

Prilagodbe rješenja

Aplikacija TravelMate razvijena je s naglaskom na prilagodljivost, kako u tehničkoj izvedbi tako i u načinu korištenja. Prilagodljivost sustava odnosi se na sposobnost aplikacije da se jednostavno prilagodi različitim korisnicima, uređajima i potrebama turističkog tržišta, bez potrebe za opsežnim promjenama u osnovnoj arhitekturi.

Prilagodba audio funkcionalnosti

Izvorno planirana funkcionalnost Text-to-Speech je prilagođena tako da koristi vanjski API servis (npr. Google Cloud Text-to-Speech) s podrškom za više jezika. Time se omogućuje da korisnici mogu birati jezik i glas vodiča, čime se rješenje bolje prilagođava međunarodnom turizmu. Audio funkcionalnost uključena je unutar prikaza svake znamenitosti, uz mogućnost pauziranja i ponovnog pokretanja opisa, što povećava pristupačnost aplikacije i čini je korisnom i za osobe s poteškoćama vida.

Prilagodba korisničkih uloga

- ▶ [6. Ispitivanje programskog rješ...](#)
- ▶ [7. Tehnologije za implementaci...](#)
- ▶ [8. Upute za puštanje u pogon](#)
- ▶ [9. Zaključak i budući rad](#)
- ▶ [A. Popis literature](#)
- ▶ [A. Dnevnik promjena dokume...](#)
- ▶ [B. Prikaz aktivnosti grupe](#)

+ Add a custom sidebar

Clone this wiki locally

<https://github.com/KristijanOr>



U sustavu su definirane dvije osnovne uloge – korisnik (User) i urednik (Editor) – no njihova su prava i sučelje prilagođeni jednostavnosti korištenja. Korisniku je omogućeno samo pregledavanje i spremanje favorita, dok urednik ima mogućnost dodavanja i uređivanja sadržaja kroz posebno uređivačko sučelje s ograničenim skupom opcija (naslov, opis, slike, radno vrijeme i dr.). Ova prilagodba omogućuje jednostavno korištenje i sprječava mogućnost pogrešaka ili neovlaštenih izmjena sadržaja. Time se zadržava funkcionalna razlika među korisnicima, ali uz naglasak na sigurnost i jednostavnost unosa podataka.

Prilagodba dizajna i responzivnost aplikacije

Posebna pažnja posvećena je dizajnu korisničkog sučelja kako bi aplikacija bila vizualno privlačna, pregledna i pristupačna širokom krugu korisnika. Sučelje je prilagođeno responzivnom prikazu, što znači da se automatski prilagođava različitim veličinama ekrana i uređajima.

Moguće nadogradnje zadatka

Poboljšanje korisničkog iskustva (UI)

- **Pametno filtriranje znamenitosti** – omogućiti filtriranje po kategorijama (povijesne, kulturne, gastronomske, prirodne atrakcije), udaljenosti ili popularnosti
- **Preporuke i personalizacija** – sustav bi mogao na temelju korisničkih interesa i prethodnih pregleda predlagati slične atrakcije

Funkcionalne nadogradnje

- **Sustav ocjenjivanja i komentiranja** – korisnici bi mogli ostavljati kratke dojmove ili ocjene za svaku atrakciju, čime se potiče interaktivnost i doprinos zajednice
- **Višejezična podrška** – omogućiti odabir jezika sučelja i sadržaja. To je posebno korisno za međunarodne turiste

Proširenje audio funkcionalnosti

- **Napredni Text-to-Speech sustav** – omogućiti izbor različitih glasova, brzina čitanja i jezika
- **Preuzimanje audio vodiča** – korisnicima bi se omogućilo preuzimanje audio datoteka za kasnije slušanje bez interneta

Sigurnost i autentifikacija

- **Dvostruka autentifikacija (2FA)** – kao dodatna zaštita korisničkih računa

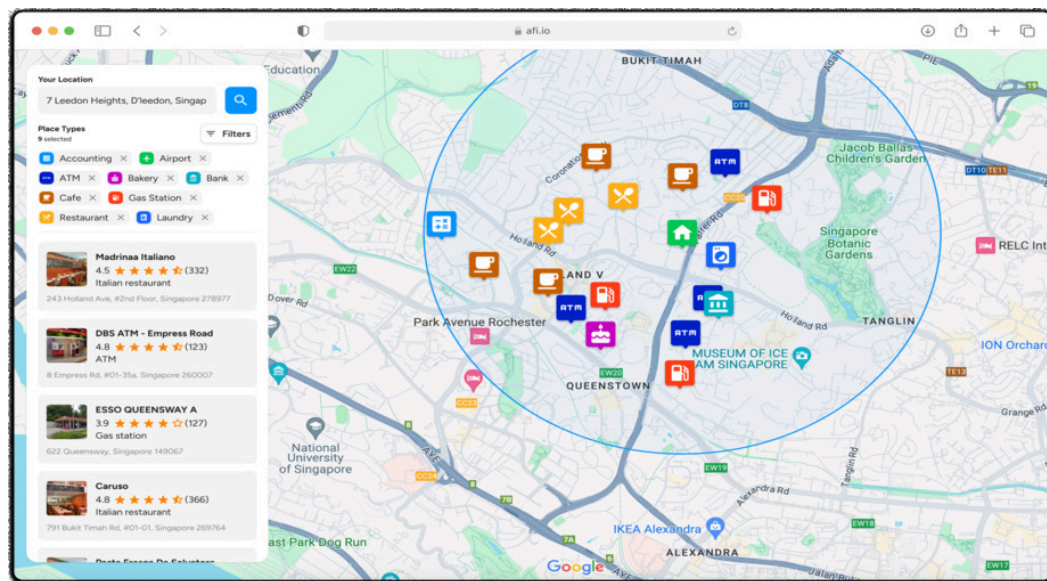
Analitika

- **Statistika posjećenosti** – omogućiti administratorima uvid u broj pregleda, najposjećenije atrakcije i najaktivnije korisnike
- **Praćenje interakcija korisnika** – analizirati koje vrste atrakcija su najtraženije radi unaprjeđenja sadržaja

Slična rješenja

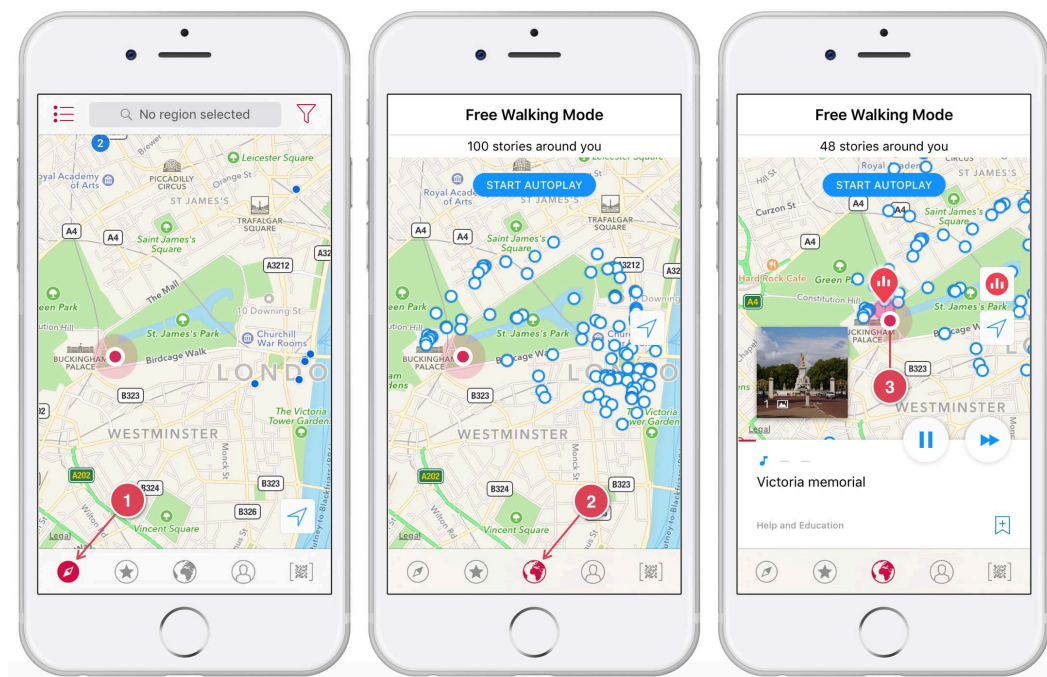
Google Maps

Google Maps je najpoznatije globalno rješenje koje, osim klasične navigacije, sadrži i funkciju „Explore Nearby“, kojom korisnici mogu istraživati znamenitosti, restorane i atrakcije u svojoj blizini. Sadrži kratke opise, fotografije i recenzije korisnika. Međutim, Google Maps nije specijaliziran za turizam – opisne informacije su ograničene, a nema audio vodič ni personalizirani sustav favorita za turističke rute, što TravelMate nudi.



izi.Travel

izi.TRAVEL je najbliže rješenje po ideji našem projektu. Radi se o mobilnoj i web aplikaciji za audio vodiče po gradovima, muzejima i znamenitostima. Korisnici mogu slušati profesionalno pripremljene priče o lokacijama, a aplikacija koristi GPS za automatsko pokretanje vodiča kada se korisnik približi određenoj točki. Iako je vrlo razvijena, većina sadržaja je ograničena na veće gradove i često zahtijeva suradnju s lokalnim institucijama, dok TravelMate cilja biti jednostavnije, otvoreno i proširivo rješenje za lokalne zajednice i manje destinacije.

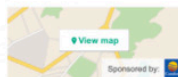


TripAdvisor

TripAdvisor je globalna platforma za planiranje putovanja koja korisnicima omogućuje pregled atrakcija, hotela i restorana uz recenzije, ocjene i preporuke drugih putnika. Iako nudi mnogo informacija, njegova je funkcionalnost usmjerena na planiranje i rezervaciju, a ne na interaktivno istraživanje lokacija na mapi. Nema integrirani sustav za slušanje opisa niti jednostavno sučelje prilagođeno samostalnom obilasku grada.

Athens Hotels

Best prices for: Sep 1 - Sep 3 1 room, 2 adults, 0 children Update



Price per night

\$0 \$450+

Amenities

- ☐ Free WiFi
- ☐ Breakfast included
- ☐ Pool
- ☐ Beach
- More

Location

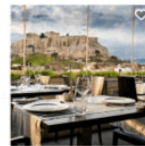
Any Distance from

Sort by: Traveler Ranked Best Value Lowest Price Distance



Royal Olympic Hotel
SAVE \$38
 Expedia.com
\$220
\$184
 View Deal
 Orbitz.com \$184
 Bookgreece.com \$183
 AMOMIA \$220
 View all 15 deals from \$183

2,973 reviews
 Free WiFi
 Breakfast included



The Athens Gate Hotel
SAVE \$59
 Expedia.com
\$303
\$244
 View Deal
 Booking.com \$243
 Hotels.com \$244
 HOTELS.INFO \$303
 View all 15 deals from \$243

3,681 reviews
 #1 Best Value of 324 hotels in Athens
 Free WiFi
 Breakfast included



Herodion Hotel
SAVE \$20
\$204

2,973 reviews

TravelMates



2. Analiza zahtjeva

Edit

New page

KristijanOresic edited this page on Nov 14, 2025 · [5 revisions](#)

Funkcionalni zahtjevi

ID zahtjeva	Opis	Prioritet	Izvor	Kriteriji prihvatanja
F-001	Aplikacija omogućuje korisnicima registraciju i prijavu putem vanjske autentifikacije (OAuth)	Visok	Zahtjev dionika	Korisnik se može registrirati i prijaviti vanjskom autentifikacijom (OAuth).
F-002	Aplikacija prikazuje znamenitosti na korisniku bliskoj lokaciji pomoću Google Maps API-ja	Visok	Zahtjev dionika	Korisniku su prikazane znamenitosti u njegovoj blizini.
F-003	Korisnik može dobiti uvid u opise, slike i korisne informacije o znamenitosti.	Srednji	Zahtjev dionika	Korisnik može vidjeti opise, slike i korisne informacije o znamenitosti.
F-004	Korisnik može spremiti znamenitosti u favorite radi bržeg pristupa.	Niski	Zahtjev dionika	Korisnik može željene znamenitosti spremiti u favorite
F-005	Aplikacija nudi mogućnost pretvaranja opisa znamenitosti u audio putem Text-to-Speech API-ja	Niski	Zahtjev dionika	Korisnik ima opciju tekst opisa znaeitosti poslušati putem Text-to-Speech API-ja
F-006	Urednici mogu dodavati nove znamenitosti i uređivati postojeće podatke.	Srednji	Zahtjev dionika	Urednici mogu dodavati nove znamenitosti i uređivati postojeće podatke.

▼ Pages13

Find a page or section...

▶ Home

▶ 1. Opis projektnog zadatka

▼ 2. Analiza zahtjeva

Funkcionalni zahtjevi

Ostali zahtjevi

Nefunkcionalni zahtjevi

Dionici

Aktori i njihovi funkcionalni zahtjevi

▶ 3. Specifikacija zahtjeva sustava

▶ 4. Arhitektura i dizajn sustava

▶ 5. Arhitektura komponenata i r...

▶ 6. Ispitivanje programskog rješ...

▶ 7. Tehnologije za implementaci...

▶ 8. Upute za puštanje u pogon

▶ 9. Zaključak i budući rad

▶ A. Popis literature

▶ A. Dnevnik promjena dokume...

▶ B. Prikaz aktivnosti grupe

+ Add a custom sidebar

Ostali zahtjevi

Nefunkcionalni zahtjevi

ID zahtjeva	Opis	Prioritet
NF-3.1	Sustav treba biti modularno dizajniran radi lakšeg održavanja i nadogradnje.	Srednji
NF-3.1.7	Sustav treba imati dovoljnu dokumentaciju.	Srednji

Clone this wiki locally

https://github.com/KristijanOr

📄

ID zahtjeva	Opis	Prioritet
NF-3.1.7.1	Uredničke funkcije moraju biti dostupne samo korisnicima s ulogom urednika.	Visok
NF-3.1.7.2	Aplikacija mora podržati povećanje broja korisnika i znamenitosti (za 10000) bez značajnog pada performansi.	Srednji
NF-3.1.7.3	Vrijeme učitavanja karte i znamenitosti ne smije prelaziti 5 sekundi.	Visok
NF-3.1.7.4	Sučelje mora biti responzivno i prilagodljivo različitim veličinama ekrana.	Visok
NF-3.1.7.5	Aplikacija mora podržavati više jezika (hrvatski, engleski).	Niski

Dionici

Dionici TravelMate aplikacije su:

- Korisnici (posjetitelji/turisti)
- Administratori(urednici)
- Razvojni tim

Članovi razvojnog tima TravelMate aplikacije su:

- Kristijan Orešić - voditelj tima
- Franko Pavlić
- Ognjen Škarić
- Nola Bonačić Dorić
- Vita Pavlović
- Nik Čolić

Aktori i njihovi funkcionalni zahtjevi

1. Korisnik registriran putem OAuth-a (turist / posjetitelj) može:
• Prijaviti se u aplikaciju putem vanjske autentifikacije (UC2) • Pregledati znamenitosti u svojoj blizini prikazane putem Google Maps API-ja (UC3) • Pregledati detalje pojedine znamenitosti, uključujući: 1. opis znamenitosti (UC4) 2. slike znamenitosti 3. povijesne informacije i zanimljive činjenice 4. radno vrijeme i cijene • Spremiti znamenitost u favorite radi bržeg pristupa (UC3.1) • Preslušati audio verziju opisa znamenitosti generiranu preko Text-to-Speech API-ja (UC4.2)
2. Korisnik registriran kao administrator (editor) može:
Urednik ima sve funkcionalnosti običnog korisnika, te dodatno može: • Dodavati nove znamenitosti (UC5.1) • Uređivati postojeće podatke znamenitosti (UC5.3) • Brisati znamenitosti (UC5.2)



3. Specifikacija zahtjeva sustava

vitapavlovic edited this page 4 hours ago · [35 revisions](#)

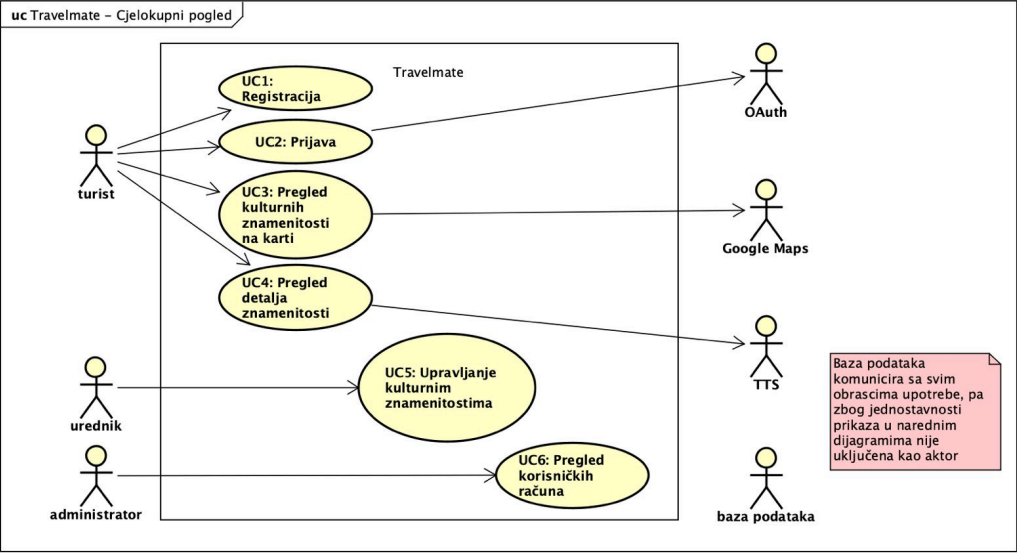
EditNew page

Obrasci uporabe

Dijagrami obrazaca uporabe

Prikazati odnos aktora i obrazaca uporabe odgovarajućim UML dijagramom. Nije nužno nacrtati sve na jednom dijagramu. Modelirati po razinama apstrakcije i skupovima srodnih funkcionalnosti.

1. Visokorazinski dijagram obrazaca uporabe cijelog sustava



▼ Pages 13

Find a page or section...

► Home

► 1. Opis projektnog zadatka

► 2. Analiza zahtjeva

▼ 3. Specifikacija zahtjeva sustava

Obrasci uporabe

Dijagrami obrazaca uporabe

1. Visokorazinski dijagram obrazaca uporabe cijelog sustava

2. Dijagram obrazaca uporabe za funkcionalnost turista

3. Dijagram obrazaca uporabe za funkcionalnost administratora i urednika

4. Dijagram obrazaca uporabe za kritične sustave i integracije

Opis obrazaca uporabe

UC1 - Registracija korisnika

UC2 - Prijava korisnika

UC2.1 - Pregled favorita

UC3 - Pregled kulturnih znamenitosti u blizini

UC3.1 - Dodavanje u favorite

UC3.2 - Dohvat lokacije

UC4 - Pregled detalja znamenitosti

UC4.1 – Učitavanje opisa i slike

UC4.2 – Slušanje audio opisa

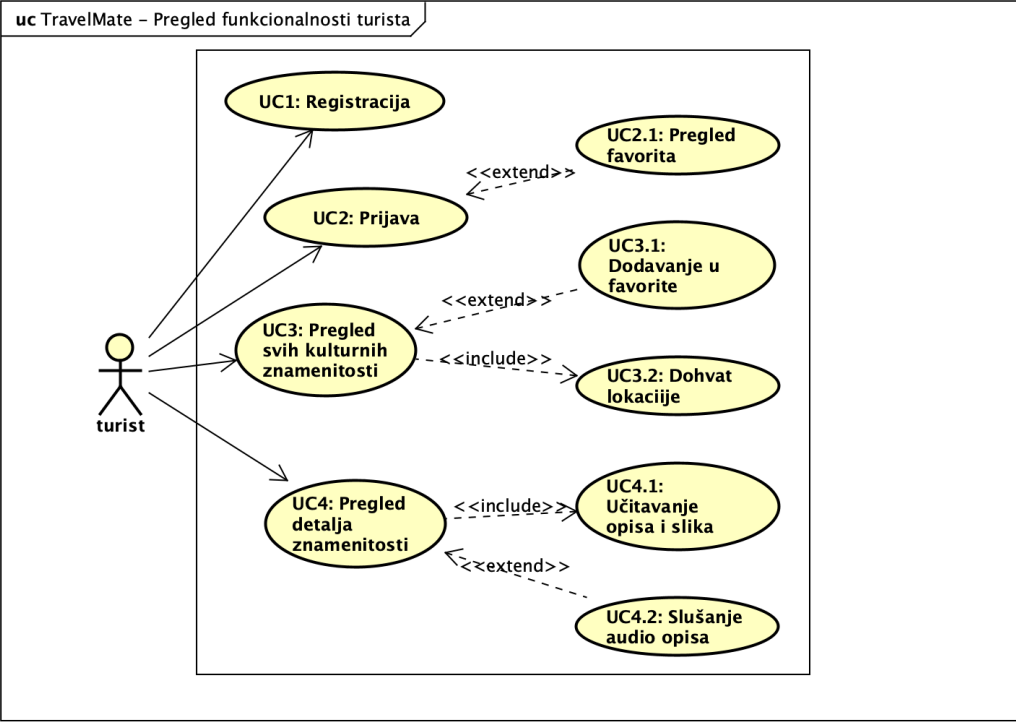
UC5.1 – Dodavanje kulturne znamenitosti

UC5.2 – Brisanje kulturne znamenitosti

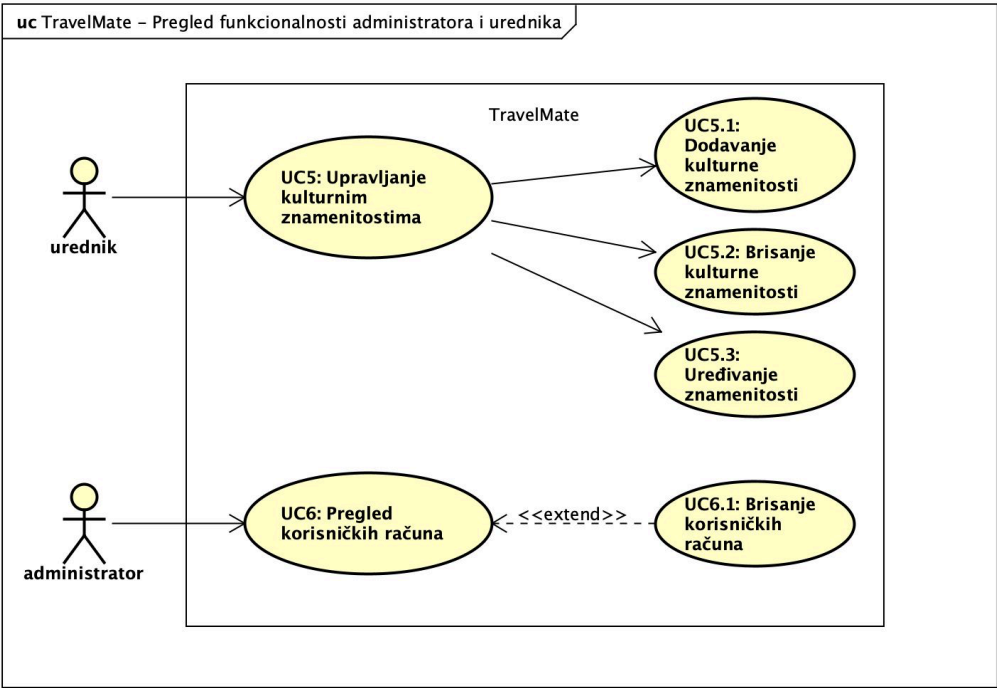
UC5.3 – Uređivanje kulturne znamenitosti

UC6 – Pregled korisničkih računa

2. Dijagram obrazaca uporabe za funkcionalnost turista



3. Dijagram obrazaca uporabe za funkcionalnost administratora i urednika



UC6.1 – Brisanje korisničkih računa

Sekvencijski dijagrami

1. Sekvencijski dijagram - Login
2. Sekvencijski dijagram - PregledavanjeMape
3. Sekvencijski dijagram - UpravljanjeKorisnicima

Provjera uključenosti ključnih funkcionalnosti u obrasce uporabe

- ▶ 4. Arhitektura i dizajn sustava
- ▶ 5. Arhitektura komponenata i r...
- ▶ 6. Ispitivanje programskog rješ...
- ▶ 7. Tehnologije za implementaci...
- ▶ 8. Upute za puštanje u pogon
- ▶ 9. Zaključak i budući rad
- ▶ A. Popis literature
- ▶ A. Dnevnik promjena dokume...
- ▶ B. Prikaz aktivnosti grupe

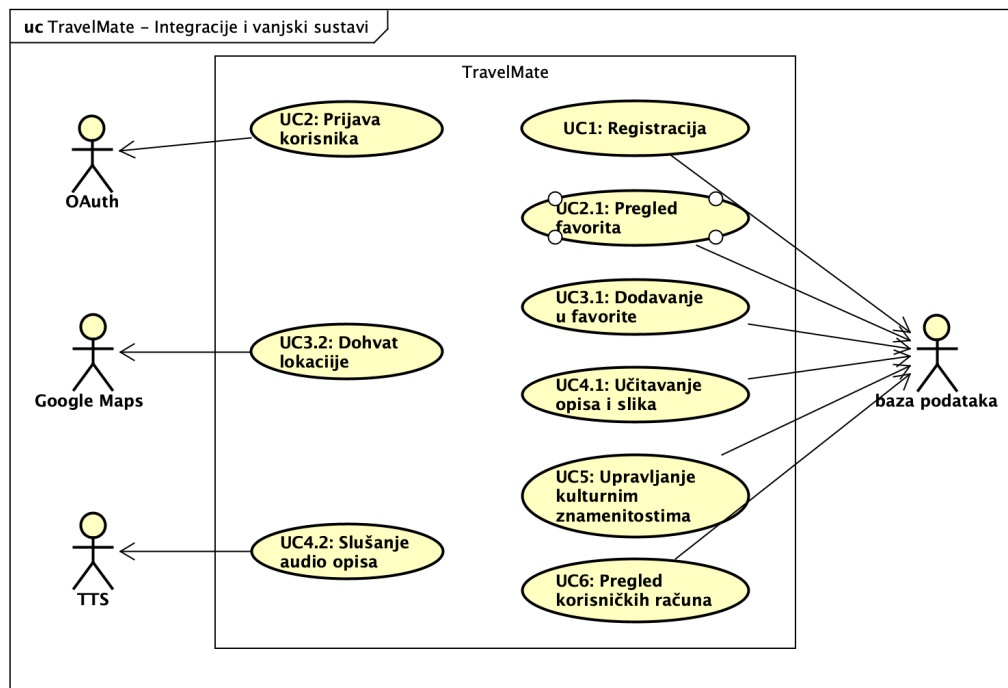
+ Add a custom sidebar

Clone this wiki locally

<https://github.com/KristijanOr>



4. Dijagram obrazaca uporabe za kritične sustave i integracije



Opis obrazaca uporabe

Funkcionalne zahtjeve razraditi u obliku obrazaca uporabe. Preporuka: Svaki obrazac je potrebno razraditi prema donjem predlošku. Ukoliko u nekom koraku može doći do odstupanja, potrebno je to odstupanje opisati i po mogućnosti ponuditi rješenje kojim bi se tijekom obrasca vratio na osnovni tijek.

UC1 - Registracija korisnika

- Glavni sudionik: turist (korisnik)
- Cilj: Korisnik kreira novi račun.
- Sudionici: baza podataka
- Preduvjet: Korisnik nije registriran u sustavu.
- Opis osnovnog tijeka:

1. Korisnik odabire opciju 'Registracija' u aplikaciji. (F-001)
2. Sustav traži unos osnovnih podataka (ime, e-mail, lozinka)
3. Korisnik potvrđuje unos
4. Sustav kreira novi korisnički račun i prikazuje poruku o uspješnoj registraciji

- Opis mogućih odstupanja:

1. E-mail već postoji -> Sustav prikazuje poruku 'Korisnik već postoji.'
2. Lozinka ne ispunjava sigurnosne uvjete -> Sustav traži ponovni unos

UC2 - Prijava korisnika

- Glavni sudionik: turist (korisnik)
- Cilj: Korisnik se prijavljuje u sustav.
- Sudionici: OAuth 2.0 (Google)
- Preduvjet: Korisnik ima registriran račun.
- Opis osnovnog tijeka:

1. Korisniku se na login stranici aplikacije nudi mogućnost prijavljivanja pomoću google računa
2. Korisnik bira opciju prijave preko google računa i unosi detalje za prijavu.
3. Sustav preko OAuth 2.0 standarda provodi autentifikaciju korisnika.
4. Korisnik je uspješno autentificiran.
5. Sustav prebacuje korisnika na naslovnu stranicu aplikacije.

• Opis mogućih odstupanja:

1. Korisnik unese pogrešne informacije, prijava nije odobrena, te prima poruku koja ga obavještava o tome.
2. Korisnik unese točne informacije računa, ali prijava nije odobrena, zato jer je taj račun blokiran od strane administratora.

UC2.1 - Pregled favorita

- Glavni sudionik: turist
- Cilj: Pregled spremljenih znamenitosti.
- Sudionici: baza podataka
- Preduvjet: Korisnik je prijavljen.
- Opis osnovnog tijeka:

1. Korisnik odabire opciju 'Favoriti'.
2. Sustav dohvaća popis spremljenih znamenitosti iz baze.
3. Popis se prikazuje korisniku.

• Opis mogućih odstupanja:

1. Baza nije dostupna -> Sustav prikazuje poruku o pogrešci.

UC3 - Pregled kulturnih znamenitosti u blizini

- Glavni sudionik: turist
- Cilj: Pregled svih kulturnih znamenitosti na karti u blizini.
- Sudionici: Google Maps API, baza podataka
- Preduvjet: Korisnik je prijavljen.
- Opis osnovnog tijeka:

1. Korisnik odabire opciju 'Pregled znamenitosti'. (F-002)
2. Sustav prikazuje znamenitosti na karti.
3. Korisnik može dodati znamenitost u favorite (UC3.1).

UC3.1 - Dodavanje u favorite

- Glavni sudionik: turist
- Cilj: Spremanje znamenitosti u favorite.
- Sudionici: baza podataka
- Preduvjet: Korisnik je prijavljen u sustav i pregledava znamenitost.
- Opis osnovnog tijeka:

1. Korisnik klikne 'Dodaj u favorite'. (F-004)
2. Sustav provjerava u bazi podataka je li određena znamenitost već označena kao omiljena.
3. Sustav uvidom podataka iz baze potvrđuje da znamenitost nije već od prije označena kao omiljena.

4. Odabrana znamenitost se sprema u bazi kao omiljena.

• Opis mogućih odstupanja:

1. Uvidom u bazu podataka sustav utvrđuje da je znamenitost već označena kao omiljena.
2. Sustav u bazi podataka odznacava odabranu znamenitost kao omiljenu.
3. Baza nije dostupna -> Sustav prikazuje poruku o grešci.

UC3.2 - Dohvat lokacije

- Glavni sudionik: sustav
- Cilj: Dohvaćanje trenutne lokacije korisnika.
- Sudionici: Google Maps API
- Preduvjet: Korisnik je omogućio pristup lokaciji.

1. Sustav šalje zahtjev za lokaciju. (F-002)
2. Korisnik dopušta pristup.
3. Sustav dohvaća lokaciju i prosljeđuje je Google Maps API-ju.
4. Prikazuje se karta sa kulturnim znamenitostima.

• Opis mogućih odstupanja:

1. Korisnik odbije pristup -> Koristi se zadana lokacija.
2. API ne odgovara -> Poruka o grešci.

UC4 - Pregled detalja znamenitosti

- Glavni sudionik: turist
- Cilj: Pregled opisa, slika i informacija.
- Sudionici: baza podataka
- Preduvjet: Korisnik je odabrao znamenitost.

1. Korisnik odabire znamenitost. (F-003)
2. Sustav uključuje UC4.1 'Učitavanje opisa i slike'.
3. Korisnik može pokrenuti UC4.2 'Slušanje audio opisa'.

• Opis mogućih odstupanja:

1. Podaci nisu dostupni -> Prikazuje se poruka o pogrešci.

UC4.1 – Učitavanje opisa i slike

- Glavni sudionik: korisnik
- Cilj: Dohvaća podatke o znamenitosti.
- Sudionici: baza podataka
- Preduvjet: Korisnik je odabrao znamenitost.

1. Sustav dohvaća opis, slike i informacije iz baze.
2. Prikazuje ih korisniku.

• Opis mogućih odstupanja:

1. Povezivanje s bazom neuspješno -> Poruka o grešci.

UC4.2 – Slušanje audio opisa

- Glavni sudionik: turist

- Cilj: Slušanje opisa putem Text-to-Speech API-ja.
- Sudionici: Text-to-Speech API
- Preduvjet: Korisnik je otvorio opis znamenitosti.

1. Korisnik odabire opciju 'Poslušaj opis'. (F-005)
2. Sustav šalje tekst na Text-to-Speech API.
3. API vraća audio datoteku.
4. Sustav reproducira audio.

• Opis mogućih odstupanja:

1. API ne odgovara -> Poruka o grešci.

###UC5 – Upravljanje kulturnim znamenitostima

- Glavni sudionik: urednik
- Cilj: Održavanje sadržaja.
- Sudionici: baza podataka
- Preduvjet: Urednik ima ovlasti.

1. Urednik odabire 'Upravljanje znamenitostima'. (F-006)
2. Sustav uključuje UC5.1, UC5.2, UC5.3.

• Opis mogućih odstupanja:

1. Baza nedostupna -> Sustav privremeno onemogućuje uređivanje.

UC5.1 – Dodavanje kulturne znamenitosti

- Glavni sudionik: urednik
- Cilj: Dodavanje nove znamenitosti.
- Sudionici: baza podataka
- Preduvjet: Urednik ima pristup sučelju za dodavanje.

1. Urednik unosi podatke o znamenitosti. (F-006)
2. Sustav sprema podatke u bazu podataka.

• Opis mogućih odstupanja:

1. Pogrešan unos -> Sustav traži ispravak.

UC5.2 – Brisanje kulturne znamenitosti

- Glavni sudionik: urednik
- Cilj: Brisanje postojeće znamenitosti. *Sudionici: baza podataka Preduvjet: Znamenitost postoji u bazi.

1. Urednik odabire znamenitost i opciju 'Obriši'. (F-006)
2. Sustav uklanja podatke iz baze.

• Opis mogućih odstupanja:

1. Baza nedostupna -> Prikazuje se poruka o grešci.

UC5.3 – Uređivanje kulturne znamenitosti

- Glavni sudionik: urednik
- Cilj: Uređivanje postojećih podataka.
- Sudionici: baza podataka

- Preduvjet: Znamenitost postoji u bazi.

1. Urednik odabire znamenitost i mijenja podatke. (F-006)
2. Sustav sprema promjene.

- Opis mogućih odstupanja:

1. Neispravan unos -> Sustav prikazuje upozorenje.

UC6 – Pregled korisničkih računa

- Glavni sudionik: administrator
- Cilj: Pregled svih korisnika u sustavu.
- Sudionici: baza podataka
- Preduvjet: Administrator je prijavljen.

1. Administrator otvara sučelje 'Korisnički računi'.
2. Sustav prikazuje popis svih korisnika.
3. Administrator može pokrenuti UC6.1 'Brisanje korisničkih računa'.

- Opis mogućih odstupanja:

1. Baza nedostupna -> Poruka o grešci.

UC6.1 – Brisanje korisničkih računa

- Glavni sudionik: Administrator
- Cilj: Brisanje korisnika iz baze.
- Sudionici: Administrator, Sustav
- Preduvjet: Administrator ima ovlasti.

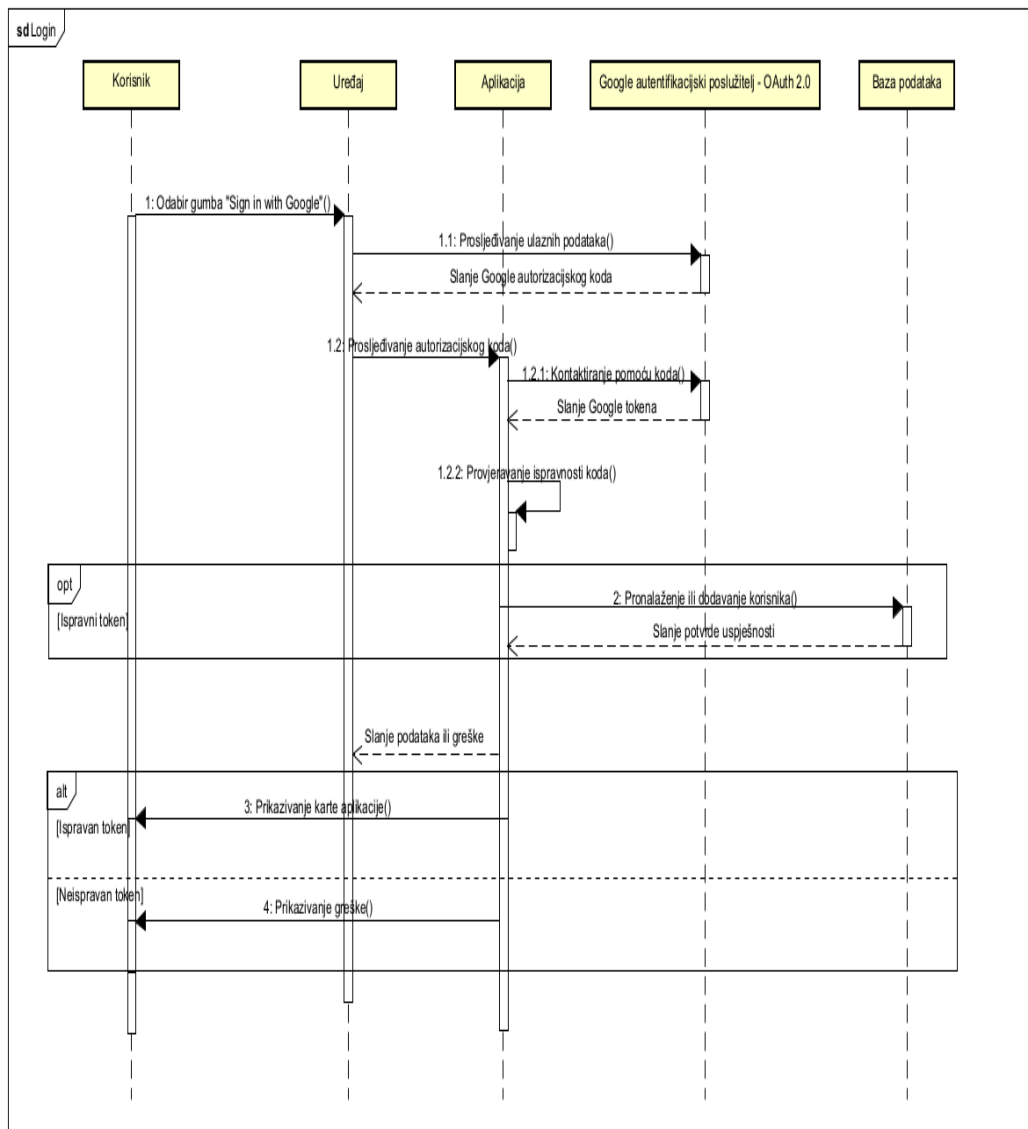
1. Administrator odabire korisnika i opciju 'Obriši korisnika'.
2. Sustav briše korisnika iz baze podataka.

- Opis mogućih odstupanja:

1. Baza nedostupna -> Poruka o grešci.

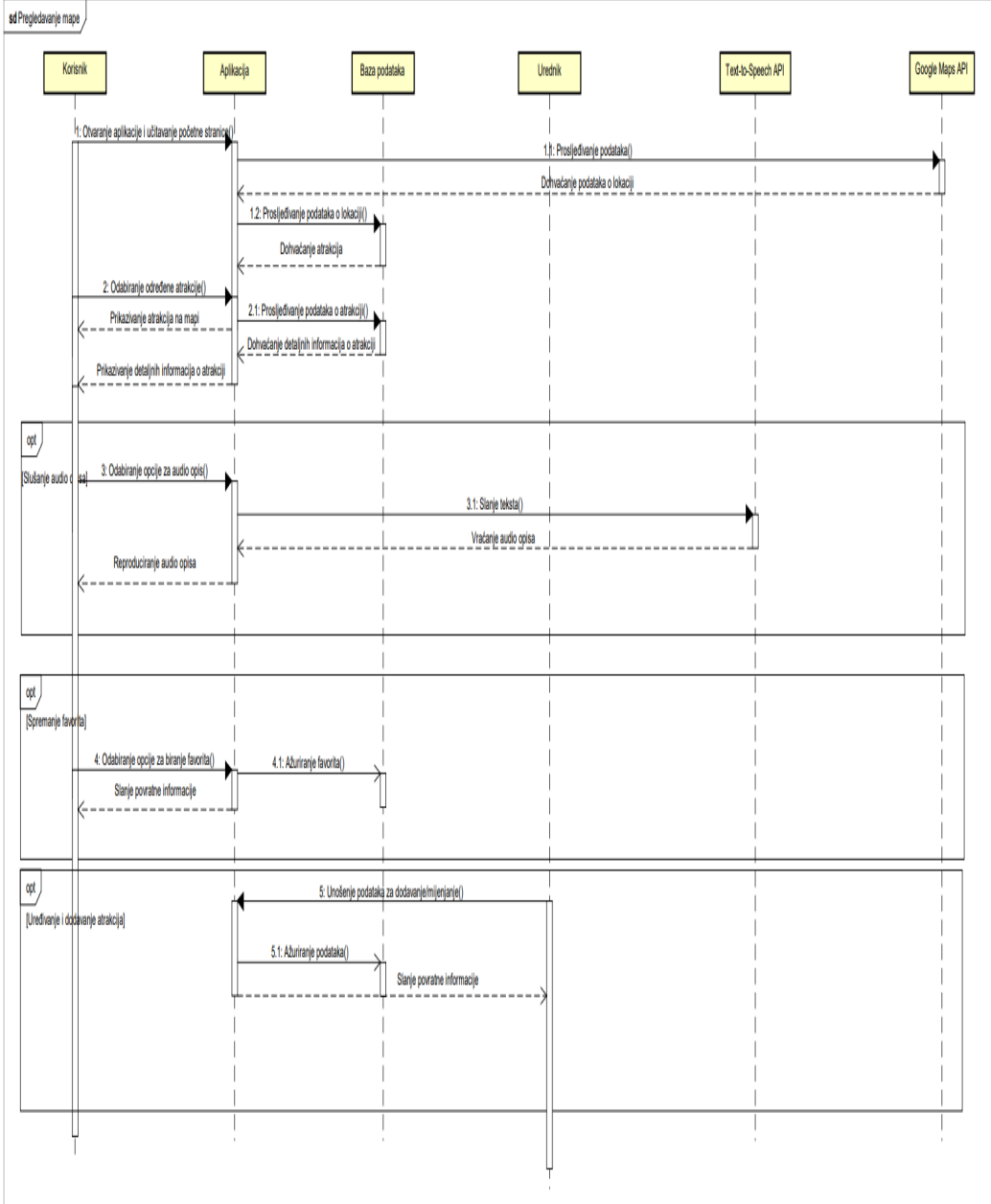
Sekvencijski dijagrami

1. Sekvencijski dijagram - Login



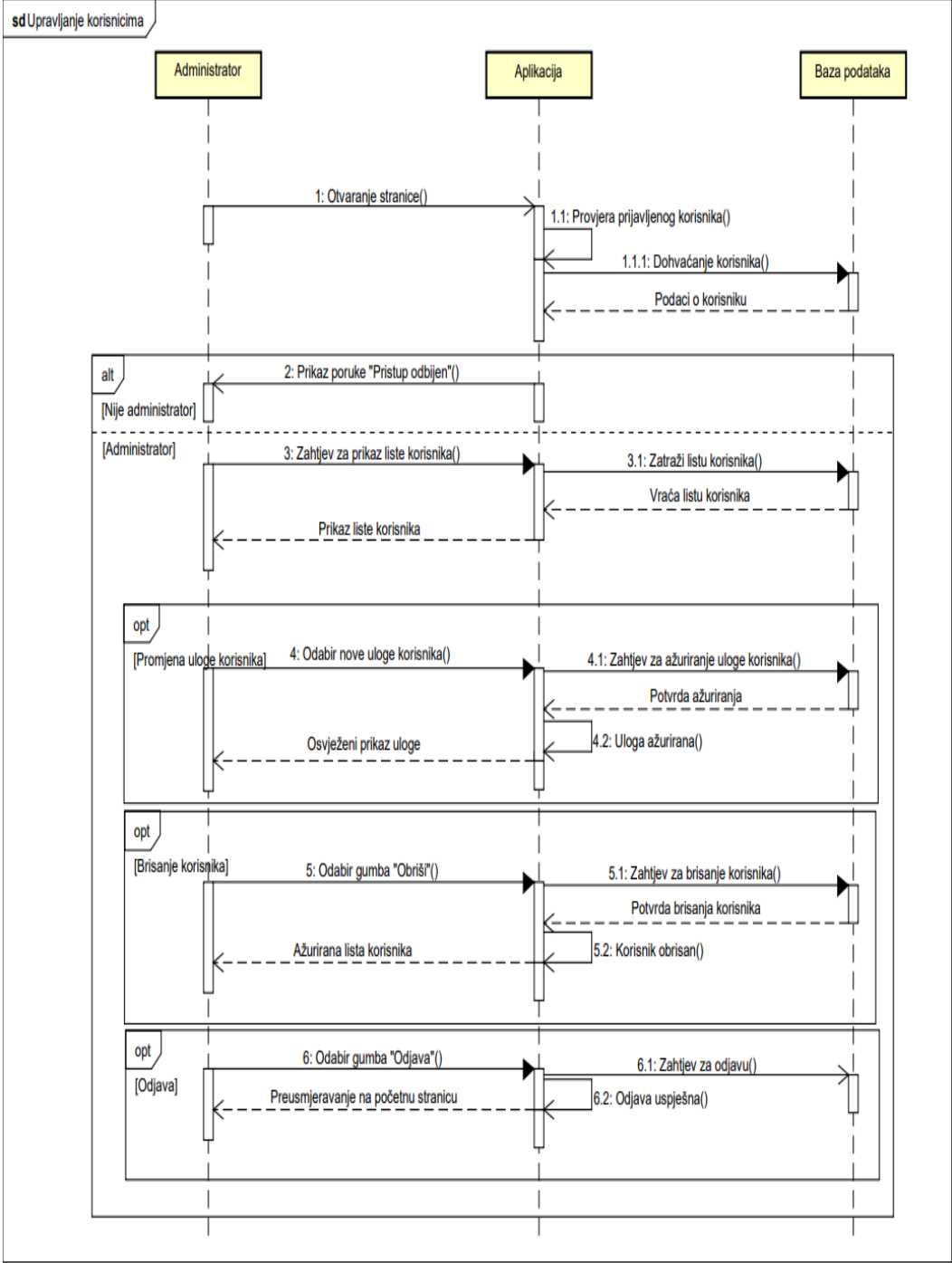
Sekvencijski dijagram Login prikazuje proces prijave korisnika u aplikaciju TravelMate pomoću korištenja Google autentifikacijskog poslužitelja - OAuth 2.0. Korisnik prvo odabire prijavu pomoću Google računa, a zatim njegov uređaj započinje autentifikacijski proces. Google poslužitelj daje autentifikacijski kod te kada ga aplikacija kontaktira pomoću tog koda, on joj šalje token, a zatim aplikacija provjerava ispravnost tog tokena. Ako je ispravan, aplikacija provjerava u bazi podataka postoji li taj korisnik te ako ne postoji ga dodaje i zatim korisniku prikazuje mapu. Ako je neispravan, prikazat će se greška. U ovom sekvencijskom dijagramu, korisnik označava i korisnika/turista i urednika.

2. Sekvencijski dijagram - PregledavanjeMape



Sekvencijski dijagram PregledavanjeMape prikazuje proces prikaza mape, svih njezinih obilježja i atrakcija te međusobnu komunikaciju različitih dijelova sustava. Korisnik otvara aplikaciju, zatim aplikacija dohvaća mapu od Google karte (Google Maps API) te pomoću baze podataka dohvaća atrakcije na mapi. Korisnik odabire atrakciju te aplikacija iz baze podataka dohvaća sve dodatne informacije o toj atrakciji. Kada korisnik odabere opciju za slušanje audio opisa, dohvaća se od Text-to-Speech sustava audio opis. Kada korisnik odabere opciju za spremanje favorita, aplikacija ažurira favorite u bazi podataka. Kada urednik dodaje ili mijenja atrakcije, aplikacija iz ažurira u bazi podataka.

3. Sekvencijski dijagram - UpravljanjeKorisnicima



Sekvencijski dijagram UpavljanjeKorisnicima prikazuje ulogu administratora i njegove mogućnosti upravljanja. Administrator otvara stranicu, zatim aplikacija provjerava je li on valjan korisnik (administrator) u bazi podataka. Ako nije administrator, aplikacija će tom korisniku vratiti poruku da je pristup odbijen. Ako je administrator, otvorit će mu se lista korisnika koju je aplikacija zatražila od baze podataka. Tada administrator može odabrati promjeniti ulogu korisniku te taj zahtjev potvrđuje i ažurira baza podataka. Također, može odabrati obrisati korisnika te se taj zahtjev također šalje i ažurira u bazi podataka. Kada se želi odjaviti, može odabrati gumb "Odjava", a onda se to i ažurira u aplikaciji i bazi podataka.

Provjera uključenosti ključnih funkcionalnosti u obrasce uporabe

| za prikazane obrasce uporabe tablično navedite koje funkcionalne zahtjeve obuhvaćaju

ID funkcijskog zahtjeva	Obrazac uporabe (Use Case)
F-001	UC1, UC2, UC2.1

ID funkcijskog zahtjeva	Obrazac uporabe (Use Case)
F-002	UC3, UC3.2
F-003	UC4, UC4.1
F-004	UC3.1, UC2.2
F-005	UC4.2
F-006	UC5, UC5.1, UC5.2, UC5.3

TravelMates

4. Arhitektura i dizajn sustava

KristijanOresic edited this page 7 minutes ago · [34 revisions](#)

EditNew page

Arhitektura sustava

Stil arhitekture: Klijent-Poslužitelj

Arhitektura korištena za aplikaciju TravelMate temelji se na modelu razdvojenog frontenda i backenda, što je standardni pristup u modernom web programiranju. Sustav slijedi klijent-poslužiteljsku arhitekturu s integriranim RESTful API komunikacijskim slojem.

Frontend (React + Vite).

Frontend je izrađen pomoću React biblioteke u Vite okruženju, što omogućuje brzi razvoj i automatsko osvježavanje stranica. React upravlja komponentama i dinamičkim prikazom podataka, a Vite optimizira produkcijsku verziju aplikacije. Frontend komunicira s backendom putem HTTP zahtjeva (Axios) prema REST API endpointima, dohvaća podatke o znamenitostima i prikazuje ih na Google karti. Sučelje je responzivno i prilagođava se mobilnim i desktop uređajima.

Backend (Node.js i Express).

Backend je implementiran u Node.js okruženju korištenjem Express frameworka.

Njegova je zadaća:

- obrada poslovne logike,
- komunikacija s bazom podataka,
- izlaganje podataka putem REST API-ja koji koristi frontend.

Backend poslužitelj dohvaća podatke iz PostgreSQL baze, filtrira ih i vraća u obliku JSON odgovora prema zahtjevima koje šalje frontend.

Baza podataka (PostgreSQL).

Podaci o znamenitostima i njihovim atributima pohranjeni su u PostgreSQL relacijskoj bazi podataka. Baza osigurava pouzdano i trajno spremanje podataka te omogućuje jednostavno izvođenje SQL upita i proširivost modela.

Razdvajanje odgovornosti

Frontend i backend djeluju kao odvojene cjeline koje međusobno komuniciraju putem REST API-ja.

Takvo razdvajanje omogućuje:

- neovisnost u razvoju i održavanju svake komponente,
- bolju organizaciju i čitljivost koda,
- jednostavno testiranje i nadogradnju sustava.

Frontend je fokusiran na korisničko iskustvo i prikaz, dok backend upravlja logikom, podacima i integracijom s vanjskim servisima.

▼ Pages13

Find a page or section...

▶ Home

▶ 1. Opis projektnog zadatka

▶ 2. Analiza zahtjeva

▶ 3. Specifikacija zahtjeva sustava

▼ 4. Arhitektura i dizajn sustava

Arhitektura sustava

Stil arhitekture: Klijent-Poslužitelj

Frontend (React + Vite)

Backend (Node.js i Express)

Baza podataka (PostgreSQL)

Razdvajanje odgovornosti

Fleksibilnost i skalabilnost

Preslikavanje na radnu platformu

Mrežni protokoli

Sklopovskoprogramski zahtjevi

Baza Podataka

Pristup podacima

Organizacija aplikacije

Frontend sloj

Backend sloj

Povezanost slojeva

Prednosti organizacije

Baza podataka

Struktura baze podataka i odnosi

CRUD operacije i prava korisnika

Glavni entiteti

Opis tablica

Users

Attractions

UserFavorites

Dijagram baze podataka

Dijagram razreda

Dinamičko ponašanje aplikacije

Fleksibilnost i skalabilnost

Ovakva arhitektura pruža fleksibilnost i skalabilnost sustava. Zahvaljujući odvojenosti slojeva, moguće je:

- jednostavno nadograditi funkcionalnosti bez potrebe za izmjenama cijelog sustava,
- proširiti aplikaciju novim servisima (npr. autentifikacija, personalizacija korisnika),
- zamijeniti pojedine komponente (npr. migrirati backend na drugu tehnologiju) bez utjecaja na ostatak sustava.

Preslikavanje na radnu platformu

Arhitektura sustava implementirana je u lokalnom razvojnom okruženju, s mogućnošću jednostavne nadogradnje na cloud platformu (npr. Render, Railway ili AWS). Trenutno se koristi Docker okruženje za izolaciju i jednostavno pokretanje podsustava. Backend i baza podataka pokreću se unutar Docker kontejnera. Frontend (React) se izvršava lokalno, ali se može lako deployati kao statička aplikacija na platformama poput Vercel ili Netlify. Odabrana platforma (lokalno + Docker) omogućuje:

- Jednostavnu konfiguraciju i testiranje tijekom razvoja,
- Reproducibilno okruženje neovisno o operacijskom sustavu,
- Mogućnost buduće migracije u cloud bez promjene arhitekture (napravljeno na Renderu)

Mrežni protokoli

Za komunikaciju između klijentske (frontend) i poslužiteljske (backend) strane koristi se HTTP protokol, koji je temeljni protokol web komunikacije. Frontend aplikacija (izrađena u Reactu) šalje HTTP zahtjeve (GET, POST) prema backend poslužitelju (izrađenom u Expressu) kako bi dohvatila ili pohranila podatke.

Komunikacija se odvija u RESTful stilu, pri čemu backend izlaže REST API krajnje točke koje frontend poziva pomoću Fetch API-ja. Svi podaci između komponenti razmjenjuju se u JSON formatu, što omogućuje jednostavnu i brzu obradu na obje strane sustava.

U budućim proširenjima sustava (npr. prikaz podataka u stvarnom vremenu, chat, dinamičko praćenje lokacije korisnika), može se razmotriti i implementacija WebSocket komunikacije za dvosmjerni prijenos podataka.

Sklopovskoprogramski zahtjevi

Aplikacija TravelMate temelji se na modernoj web arhitekturi te ne zahtijeva posebno prilagođeno sklopovlje. Sustav je zamišljen kao lagana web aplikacija, čija se komponenta frontend izvršava u korisničkom web pregledniku, dok se backend i baza podataka pokreću na poslužitelju ili lokalnom razvojnom okruženju.

Programsko okruženje:

- Operativni sustavi: aplikacija je potpuno višestruko platformna i može se pokretati na Windows, macOS ili Linux sustavima.
- Backend: zahtijeva instaliran Node.js (verzija 18 ili novija) i npm za upravljanje ovisnostima.
- Frontend: razvijen u Reactu, izvršava se u bilo kojem modernom pregledniku lokalno (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari).
- Baza podataka: koristi PostgreSQL sustav, koji se zasad pokreće preko Docker kontejnera, kasnije preko Rendera.

UML dijagrami stanja
UML dijagrami aktivnosti
▸ 5. Arhitektura komponenata i r...
▸ 6. Ispitivanje programskog rješ...
▸ 7. Tehnologije za implementaci...
▸ 8. Upute za puštanje u pogon
▸ 9. Zaključak i budući rad
▸ A. Popis literature
▸ A. Dnevnik promjena dokume...
▸ B. Prikaz aktivnosti grupe

+

Add a custom sidebar

Clone this wiki locally

https://github.com/KristijanOr

Baza Podataka

Sustav koristi relacijsku bazu podataka PostgreSQL kao glavno spremište podataka. Ova tehnologija je odabrana zbog svoje pouzdanosti, podrške za kompleksne relacije i standardnog SQL jezika, što je idealno za strukturirane podatke poput znamenitosti, korisnika ili drugih entiteta koji se mogu pojaviti u budućim proširenjima aplikacije.

Pristup podacima

Backend aplikacija (Node.js + Express) koristi biblioteku pg za komunikaciju s bazom podataka putem SQL upita. Podaci se dohvaćaju i vraćaju frontendu u JSON formatu putem REST API-ja, što omogućuje jednostavnu integraciju i prikaz podataka u React sučelju.

Organizacija aplikacije

Aplikacija TravelMate organizirana je prema principima višeslojne web arhitekture, s jasno razdvojenim frontend i backend dijelom. Takva organizacija omogućuje neovisnost u razvoju, jednostavnije održavanje, bolju skalabilnost te mogućnost proširenja aplikacije dodatnim servisima.

Frontend sloj

Frontend predstavlja korisnički dio sustava i izrađen je pomoću React biblioteke. Zadužen je za prikaz podataka, interakciju s korisnikom i komunikaciju s backendom putem REST API-ja.

Glavne funkcionalnosti frontenda uključuju:

- prikaz karte znamenitosti pomoću Google Maps API-ja
- prikaz opisa, slika i dodatnih informacija o lokacijama
- autentifikaciju i upravljanje korisničkim sesijama (JWT, Google prijava)
- spremanje omiljenih znamenitosti i prikaz korisničkog sadržaja

Struktura React aplikacije obuhvaća

- src/pages – prikazi aplikacije (MapPage, UserPage, AdminPage, ChooseRole)
- src/styles – CSS datoteke za stilizaciju i responzivnost
- App.jsx i main.jsx – glavni dio aplikacije i konfiguracija ruta

Frontend koristi React Router DOM za navigaciju, Axios za komunikaciju s poslužiteljem te jwt-decode za upravljanje sesijama. Modularni pristup komponentama omogućuje jednostavnu nadogradnju korisničkog sučelja i ponovno korištenje postojećih elemenata.

Backend sloj

Backend je izrađen u Node.js okruženju uz korištenje Express okvira. Zadužen je za obradu korisničkih zahtjeva, autentifikaciju i rad s bazom podataka. Backend sloj izlaže REST API koji omogućuje dohvat, dodavanje i uređivanje znamenitosti.

Struktura backenda:

- server.js – pokretanje Express poslužitelja i konfiguracija aplikacije
- routes/attractions.js – rute za upravljanje znamenitostima
- database.sql i initDB.js – definicija i inicijalizacija baze podataka
- python/app.py – pomoćni mikroservis izrađen u Flasku za obradu podataka i integraciju s vanjskim API-jevima (npr. Text-to-Speech)

Za pohranu podataka koristi se PostgreSQL baza. Autentifikacija korisnika provodi se pomoću JWT tokena i Google OAuth 2.0, dok su sigurnosne komponente implementirane uz pomoć bcrypt i express-session modula. Sustav podržava pokretanje putem Docker Compose konfiguracije, koja povezuje Node i Python servise u jedno okruženje.

Povezanost slojeva

Komunikacija između slojeva odvija se putem HTTP zahtjeva. Frontend šalje zahtjeve prema Express poslužitelju, koji dohvaća ili mijenja podatke u bazi te vraća odgovor u JSON formatu. Za generiranje audio vodiča, Node backend koristi Flask mikroservis koji poziva Google Text-to-Speech API i vraća obrađeni audio sadržaj. Na taj način postiže se fleksibilan sustav u kojem su komponente međusobno povezane, ali funkcionalno neovisne.

Prednosti organizacije

- razdvajanje slojeva omogućuje neovisni razvoj frontenda, backenda i pomoćnih servisa
- modularnost i proširivost – novi servisi (npr. preporuke znamenitosti) mogu se lako integrirati
- sigurnost i pouzdanost – primjena JWT i OAuth 2.0 mehanizama osigurava kontrolu pristupa i zaštitu korisničkih podataka
- skalabilnost – svaka komponenta sustava može se razvijati i nadograđivati zasebno, što olakšava održavanje i budući razvoj.

Baza podataka

Za izradu web aplikacije TravelMate koristi se PostgreSQL baza podataka pokrenuta unutar Docker okruženja. Ovakav pristup omogućuje jednostavno dijeljenje i pokretanje baze na različitim uređajima, što olakšava razvoj i testiranje.

U razvojnoj fazi baza se koristi lokalno preko Dockera, dok će se u produkciji baza hostati na serveru. Backend aplikacija bit će deployana na Render, što omogućuje trajnu pohranu podataka, skalabilnost i pouzdanu dostupnost za sve korisnike. Docker razvojno okruženje tako služi samo za lokalni razvoj i testiranje.

Struktura baze podataka i odnosi

Baza uključuje različite vrste relacija između entiteta:

- N:N -> Jedan korisnik može imati više favorita, a jedna atrakcija može biti favorit za više korisnika. Ova veza je modelirana kroz pomoćnu tablicu userFavorites koja povezuje entitete users i attractions.
- 1:1..N i 1..N:1 -> Svaka atrakcija ima barem jednu sliku, a svaka slika pripada točno jednoj atrakciji. Ova veza omogućava pohranu više fotografija po atrakciji za galeriju ili prikaz detalja.
- 1:0..N i 0..N:1 -> Jedan administrator može dodati više atrakcija u bazu podataka, ali ta atrakcija može biti dodana samo od strane jednog administratora. Ovo omogućava praćenje tko je kreirao atrakciju i tko je odgovoran za njen sadržaj.

CRUD operacije i prava korisnika

Pristup bazi je organiziran prema CRUD (Create, Read, Update, Delete) operacijama, koje su prilagođene različitim korisničkim ulogama:

- Korisnici (role = 'user') imaju pristup pregledavanju (Read) informacija o atrakcijama i mogu dodavati atrakcije u svoje favorite (Create). Također mogu pregledavati slike i audio zapise atrakcija.

- Administratori (role = 'admin') imaju proširenu razinu pristupa, uključujući dodavanje (Create) novih atrakcija, ažuriranje (Update) podataka atrakcija, slika i audio zapisa, te brisanje (Delete) atrakcija. Administratori također mogu upravljati korisničkim računima i njihovim ulogama.
- Opće pravilo: prava pristupa su implementirana tako da korisnici mogu raditi samo ono što je dopušteno njihovom ulogom, čime se osigurava sigurnost i integritet podataka u bazi.

Glavni entiteti

- Korisnici (users): Sadrži podatke o korisnicima aplikacije, uključujući obične korisnike i administratore. Tablica pohranjuje ime, prezime, e-mail i ulogu korisnika, što omogućava kontrolu pristupa različitim funkcionalnostima aplikacije.
- Atrakcije (attractions): Sadrži informacije o turističkim atrakcijama, uključujući ime, opis, lokaciju (geografske koordinate), web stranicu, povijesne podatke, zanimljivosti, radno vrijeme i informacije o cijeni. Svaka atrakcija može biti dodana od strane određenog korisnika (administrator).
- Favoriti korisnika (userFavorites): Povezuje korisnike s njihovim omiljenim atrakcijama. Ova pomoćna tablica omogućava modeliranje (N:N) odnosa između korisnika i atrakcija te osigurava da isti korisnik ne može označiti istu atrakciju više puta kao favorit.

Opis tablica

Users

Atribut	Tip podatka	Opis varijable
idUser	INT	Jedinstveni identifikator (primarni ključ)
firstName	VARCHAR(20)	Ime korisnika
lastName	VARCHAR(20)	Prezime korisnika
email	VARCHAR(30)	Email korisnika
role	VARCHAR(10)	Uloga korisnika

Attractions

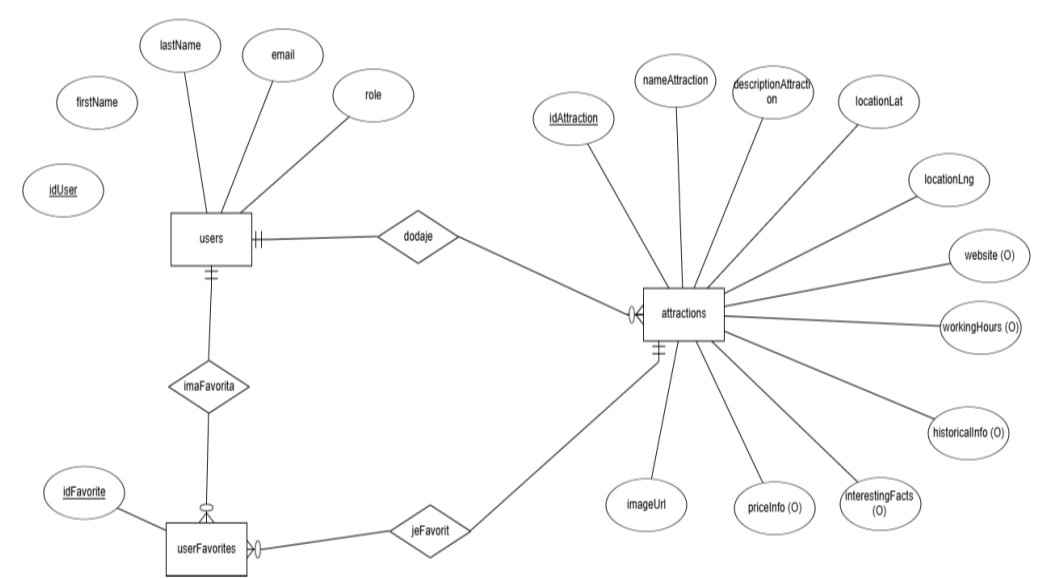
Atribut	Tip podatka	Opis varijable
idAttraction	INT	Jedinstveni identifikator (primarni ključ)
nameAttraction	VARCHAR(50)	Naziv atrakcije
descriptionAttraction	VARCHAR(500)	Opis atrakcije
locationLat	DOUBLE PRECISION	Lateralna koordinata lokacije
locationLng	DOUBLE PRECISION	Longitudinalna koordinata lokacije
website	VARCHAR(100)	Web stranice objekta na lokaciji
historicalInfo	VARCHAR(1000)	Povijesne činjenice o lokaciji
interestingFacts	VARCHAR(1000)	Zanimljivosti o lokaciji
workingHours	VARCHAR(100)	Radno vrijeme

Atribut	Tip podatka	Opis varijable
priceInfo	VARCHAR(50)	Informacije o cijenama na toj lokaciji
imageUrl	VARCHAR(200)	URL slike znamenitosti
idUser	INT	ID admina koji je dodao lokaciju (strani ključ)

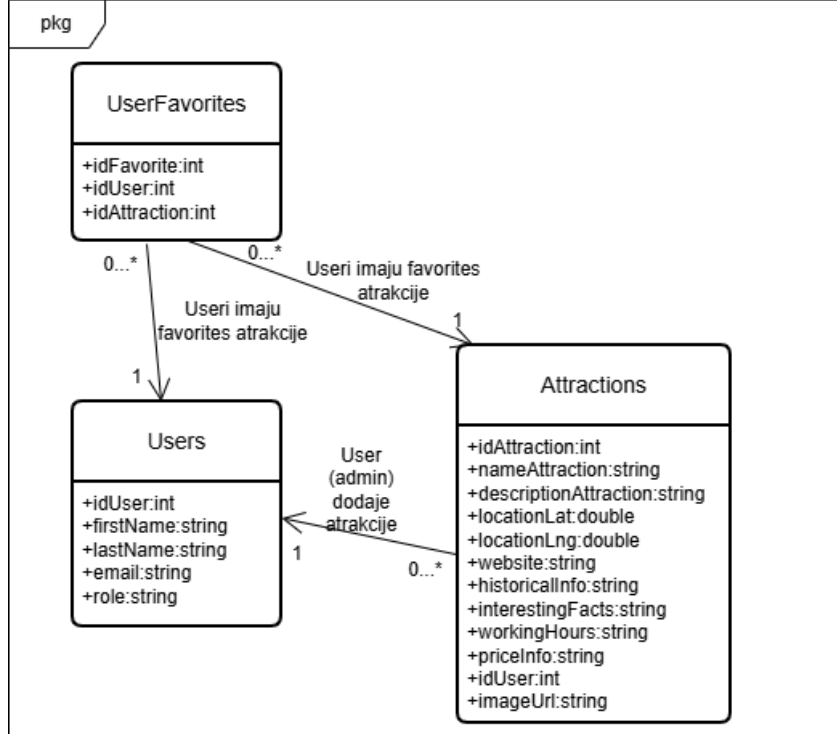
UserFavorites

Atribut	Tip podatka	Opis varijable
idFavorite	INT	ID favorita (primarni ključ)
idUser	INT	ID korisnika (strani ključ)
idAttraction	INT	ID atrakcije (strani ključ)

Dijagram baze podataka



Dijagram razreda



Users: Atributi: `idUser`, `firstName`, `lastName`, `email`, `role`. Odnosi: Može dodavati znamenitosti u favorite(1:N), ako je korisnik admin može dodavati nove znamenitosti(1:N).

Attractions: Atributi: `idAttraction`, `nameAttraction`, `descriptionAttraction`, `locationLat`, `locationLng`, `website`, `historicalInfo`, `interestingFacts`, `workingHours`, `priceInfo`, `imageUrl`, `idUser`. Odnosi: povezana s **Users**(0..N:1) gdje administratori mogu dodavati znamenitosti.

UserFavorites: Atributi: `idFavorite`, `idUser`, `idAttraction`. Odnosi: povezana s **Users**(0..N:1), što znači da taj id Favorita može biti namijenjen samo jednom korisniku, a korisnik može imati 0 ili više favorita, također povezana s **Attractions**(0..N:1), odnosno jedan id favorita može biti namijenjen jednoj atrakciji, a jedna atrakcija može biti favorit kod više korisnika.

Dinamičko ponašanje aplikacije

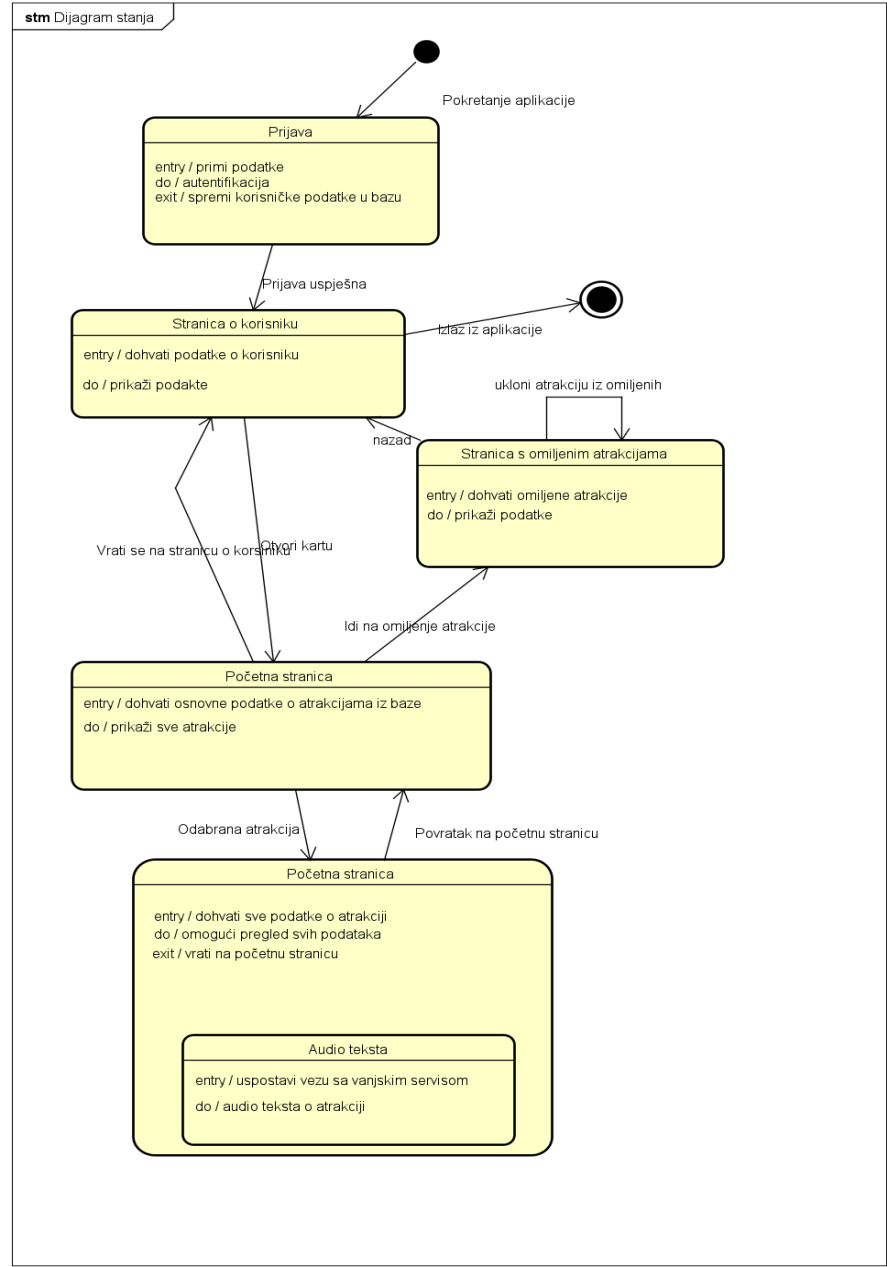
Dinamičko ponašanje aplikacije odnosi se na način na koji objekti u sustavu evoluiraju kroz vrijeme, uključujući prijelaze između različitih stanja. To uključuje aktivnosti, događaje, odluke i interakcije unutar aplikacije. UML dijagrami stanja omogućuju vizualizaciju tih promjena i olakšavaju razumijevanje dinamike sustava.

Razumijevanje promjena stanja neophodno je za pravilno funkcioniranje aplikacije jer pruža uvid u interakcije među objektima, komponentama i korisnicima tijekom rada sustava. Korištenjem UML dijagrama stanja i aktivnosti moguće je vizualizirati prijelaze i stanja objekata, identificirati potencijalne probleme, osigurati točnu implementaciju te poboljšati komunikaciju među članovima tima.

Zadatak:

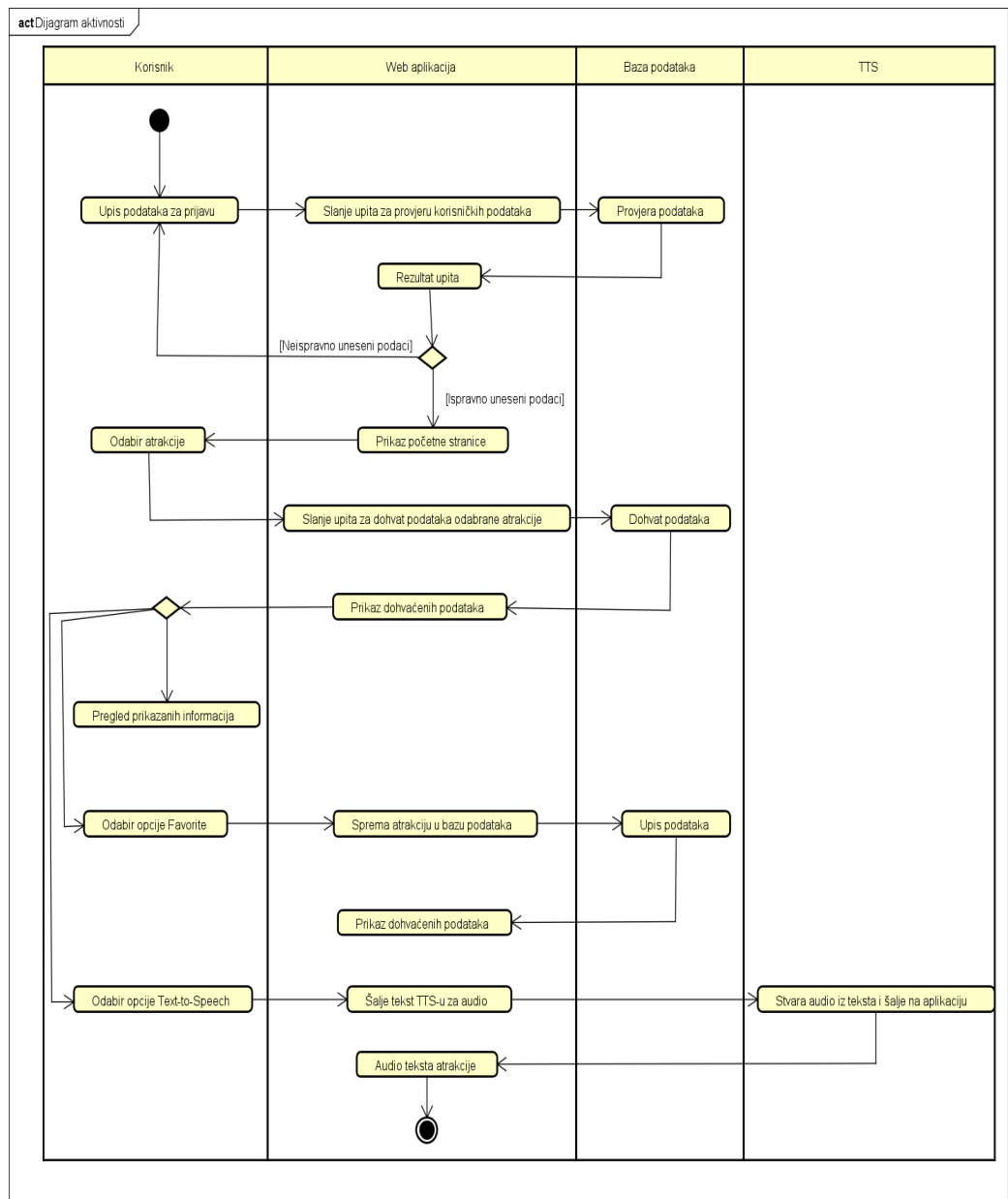
- Dokumentirajte dva dinamička dijagrama koji prikazuju značajne ili neke složene procese u aplikaciji. Registracija i prijava korisnika nisu ključni procesi u ovom kontekstu, stoga se usmjerite na specifične, značajne procese.

UML dijagrami stanja



U dijagramu stanja imamo više stanja Prijava, Stranica o korisniku, Stranica s omiljenim atrakcijama, i dvije Početne stranice. Prijelaz iz jednog u drugog stanja se događa kad korisnik pritisne određeni gumb u aplikaciji.

UML dijagrami aktivnosti



- Korisnik: upisuje podatke za prijavu, odabire atrakciju, pregledava prikazane informacije, odabire opciju Favorite i odabire opciju Text-to-Speech
- Web-aplikacija: šalje upit za provjeru korisničkih podataka, pregledava rezultat upita, prikazuje početnu stranicu, šalje upit za dohvat podataka odabrane atrakcije i prikazuje te podatke kad ih dohvati, sprema atrakciju u bazu podataka, šalje tekst TTS-u za audio i audio tekst atrakcije.
- Baza podataka: u njoj se dešavaju provjere podataka, dohvat i upis
- TTS: stvara audio iz teksta

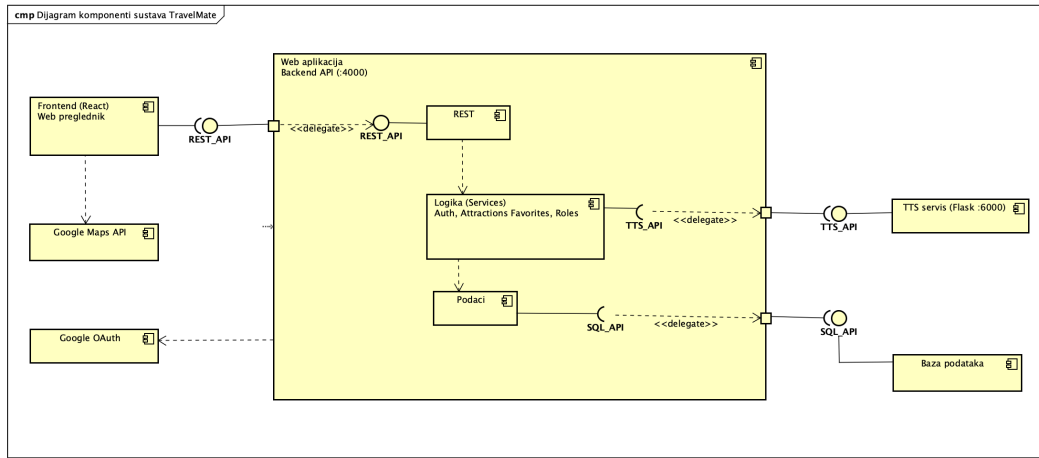
5. Arhitektura komponenata i razmještaja

Nola Bonačić-Dorić edited this page 4 hours ago · [17 revisions](#)

Edit

New page

Dijagram komponenata



Dijagram komponenata prikazuje logičku strukturu sustava TravelMate i međusobne odnose između glavnih funkcionalnih dijelova aplikacije. Sustav je podijeljen na frontend dio koji se izvršava u web pregledniku korisnika te backend dio koji pruža REST API usluge. Frontend komponenta, implementirana u React okviru, komunicira s backend API-jem putem REST sučelja. Backend aplikacija sadrži sloj REST kontrolera koji prima zahtjeve s frontenda i proslijeđuje ih servisnoj logici.

Sloj logike (Services) odgovoran je za obradu poslovnih funkcionalnosti sustava, uključujući autentifikaciju korisnika, dohvat i upravljanje znamenitostima, spremanje favorita te upravljanje korisničkim ulogama. Ovaj sloj komunicira s komponentom za pristup podacima (Podaci), koja je zadužena za komunikaciju s bazom podataka putem SQL sučelja.

Za funkcionalnost pretvaranja teksta u govor koristi se zasebni TTS servis implementiran u Flask okviru, s kojim backend komunicira putem REST sučelja. Time je omogućena modularnost sustava i neovisno održavanje audio funkcionalnosti.

Sustav koristi i vanjske servise, poput Google Maps API-ja za prikaz interaktivne karte na frontend strani te Google OAuth za autentifikaciju korisnika, čime se osigurava jednostavna i sigurna prijava.

Ovakva arhitektura omogućuje jasnu razdvojenost odgovornosti između komponenata, lakše održavanje sustava te mogućnost budućih nadogradnji i zamjene pojedinih dijelova bez utjecaja na ostatak aplikacije.

▼ Pages 13

Find a page or section...

▶ Home

▶ 1. Opis projektnog zadatka

▶ 2. Analiza zahtjeva

▶ 3. Specifikacija zahtjeva sustava

▶ 4. Arhitektura i dizajn sustava

▼ 5. Arhitektura komponenata i r...

Dijagram komponenata

Dijagram razmještaja

▶ 6. Ispitivanje programskog rješ...

▶ 7. Tehnologije za implementaci...

▶ 8. Upute za puštanje u pogon

▶ 9. Zaključak i budući rad

▶ A. Popis literature

▶ A. Dnevnik promjena dokume...

▶ B. Prikaz aktivnosti grupe

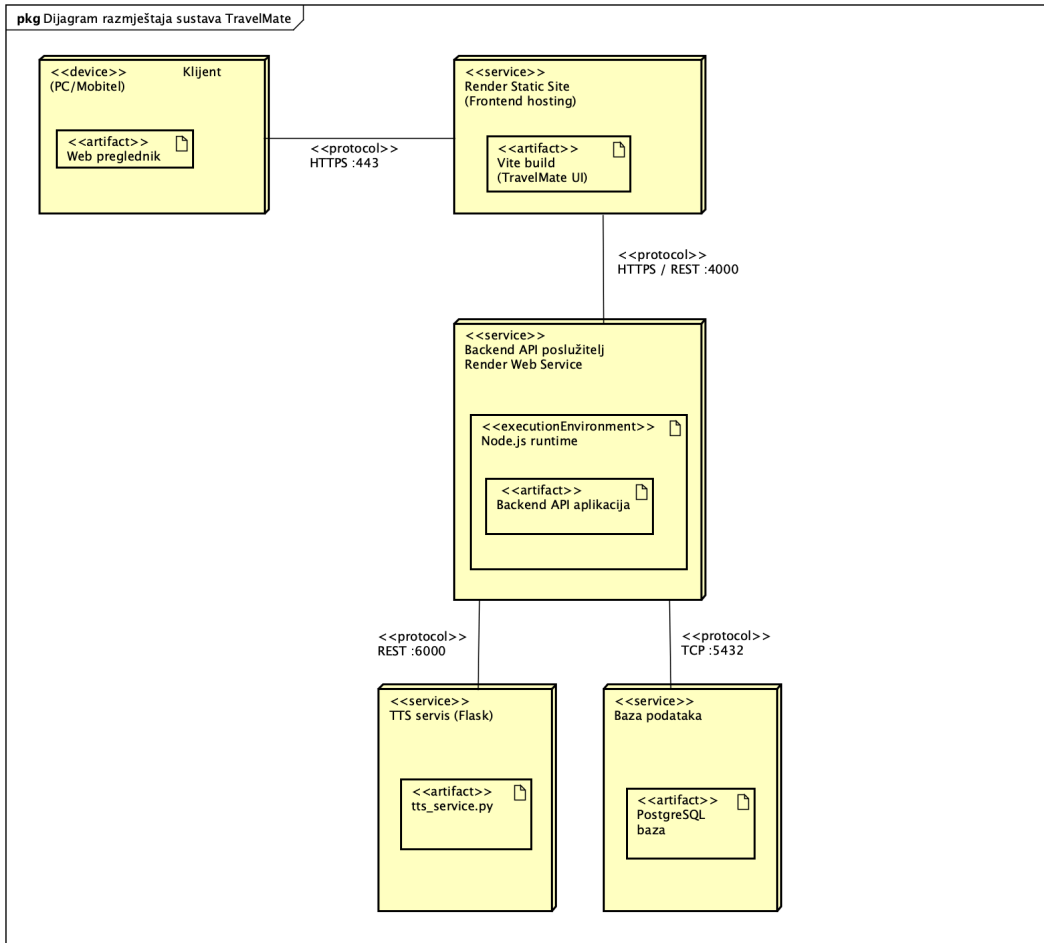
+ Add a custom sidebar

Clone this wiki locally

<https://github.com/KristijanOr>

📄

Dijagram razmještaja



Dijagram razmještaja prikazuje fizičku i virtualnu raspodjelu aplikacijskih komponenta sustava TravelMate unutar infrastrukture. Sustav je organiziran kao višeslojna web aplikacija s odvojenim frontend i backend dijelovima te dodatnim servisima.

Korisnici pristupaju sustavu putem web preglednika na osobnim računalima ili mobilnim uređajima. Frontend aplikacija implementirana je kao statička web aplikacija te je hostana na Render Static Site servisu, koji služi za isporuku Vite buildanih React datoteka korisnicima putem HTTPS protokola na portu 443.

Frontend aplikacija komunicira s backend dijelom sustava putem HTTPS REST zahtjeva na portu 4000. Backend aplikacija pokrenuta je kao Render Web Service te se izvršava unutar Node.js okruženja. Backend API obrađuje zahtjeve vezane uz autentifikaciju korisnika, upravljanje znamenitostima, favorite i korisničke uloge.

Za funkcionalnost pretvaranja teksta u govor koristi se zaseban TTS servis implementiran u Flask okviru, koji je pokrenut kao samostalni servis. Backend aplikacija s TTS servisom komunicira putem REST protokola na portu 6000.

Podaci sustava pohranjuju se u PostgreSQL bazi podataka, s kojom backend komunicira putem TCP veze na portu 5432. U razvojnome okruženju baza podataka se pokreće unutar Docker kontejnera kako bi svi članovi tima imali jednaku strukturu baze podataka, dok su sami podaci lokalni za svakog korisnika.

Vanjski servisi poput Google Maps API-ja i Google OAuth-a koriste se putem sigurnih HTTPS veza te nisu dio infrastrukture sustava, već se koriste kao cloud usluge trećih strana.

Ovakav način razmještaja omogućuje skalabilnost sustava, jednostavno održavanje pojedinih servisa te fleksibilnost u budućem proširenju aplikacije.



Test 02: Funkcionalnost - Dodavanje Atrakcija (POST)

Funkcionalnost koju testiramo:

Provjera rute za unos novih lokacija koja zahtijeva editor role.

Ispitni slučaj 1: Uspješno dodavanje s editor tokenom

- Ulaz:

```
[
  {
    "name": "Split",
    "description": "Opis",
    "location_lat": 43.5,
    "location_lng": 16.4
  }
]
```



- Rezultat: (200 OK)
- Tijelo odgovora:

```
{
  "idAttraction": 1,
}
```



Status: PROLAZ

Ispitni slučaj 2: Validacija tipova podataka (String-to-Float)

- Opis: Slanje koordinata kao stringova ("44.1") kako bi se provjerila automatska konverzija.
- Rezultat: Sustav uspješno obrađuje zahtjev bez 400 Bad Request.

Status: PROLAZ

Ispitni slučaj 3: Odbijanje zahtjeva zbog nepotpunih podataka

- Opis: Pokušaj slanja zahtjeva bez obaveznog polja name
- Rezultat: (400 Bad Request)
- Tijelo odgovora:

```
{
  "error": "Name, description and coordinates are required"
}
```



Status: PROLAZ



Test 03: Funkcionalnost - Provjera Emaila

Funkcionalnost koju testiramo:

Ispitni slučaj 1: Email već postoji u bazi

Status: PROLAZ

Ispitni slučaj 2: Odbijanje zahtjeva bez parametara

Status: PROLAZ

Test 04: Funkcionalnost - Autentifikacija (JWT)

Funkcionalnost koju testiramo:

Ispitni slučaj 1: Odbijanje neispravnog tokena

Status: PROLAZ

Test 05: Funkcionalnost - Upravljanje korisnicima (Brisanje)

Funkcionalnost koju testiramo:

Ispitni slučaj 1: Uspješno brisanje

Status: PROLAZ

Ispitni slučaj 2: Korisnik ne postoji (idempotentnost)

Status: PROLAZ

Test 06: Funkcionalnost - Pogreške i Rutiranje

Funkcionalnost koju testiramo:

Ispitni slučaj 1: Nepostojeća ruta

Status: PROLAZ

Ispitni slučaj 1: Kritični pad baze podataka

Status: PROLAZ

Ispitivanje sustava

Test 01: Registracija korisnika

Funkcionalnost koju testiramo:

Ispitni slučaj:

Ulazni podaci:

Očekivani rezultati:

Postupak provođenja ispitivanja:

Dobiveni rezultati:

Status: PROLAZ

Test 02: Validacija – prazna forma

Funkcionalnost koju testiramo:

Ispitni slučaj:

Ulazni podaci:

Očekivani rezultati:

Postupak provođenja ispitivanja:

Dobiveni rezultati:

Status: PROLAZ

Funkcionalnost koju testiramo:

Provjera rute /check-email koja se koristi za "real-time" validaciju dostupnosti email adrese.

Ispitni slučaj 1: Email već postoji u bazi

- Ulaz:

```
{  
  "email": "test@fer.hr"  
}
```



- Rezultat: (200 OK)
- Tijelo odgovora:

```
{  
  "exists": true  
}
```



Status: PROLAZ

Ispitni slučaj 2: Odbijanje zahtjeva bez parametara

- Ulaz:

```
{}
```



- Rezultat: (400 Bad Request):
- Tijelo odgovora:

```
{  
  "error": "Email je obavezan"  
}
```



Status: PROLAZ

Test 04: Funkcionalnost - Autentifikacija (JWT)

Funkcionalnost koju testiramo:

Provjera middleware zaštite koja sprječava neovlašten pristup podacima.

Ispitni slučaj 1: Odbijanje neispravnog tokena

- Putanja: /me (dohvat profila)
- Ulaz: token=neispravan-token-123
- Rezultat: (401 Unauthorized)
- Tijelo odgovora:

Test 03: Zabrana pristupa admin funkcijama

Funkcionalnost koju testiramo:

Ispitni slučaj:

Ulazni podaci:

Očekivani rezultati:

Postupak provođenja ispitivanja:

Dobiveni rezultati:

Status: PROLAZ

Test 04: Prikaz mape i favorita

Funkcionalnost koju testiramo:

Ispitni slučaj:

Očekivani rezultati:

Postupak provođenja ispitivanja:

Dobiveni rezultati:

Status: PROLAZ

Test 05: Prikaz mobilne verzije (responzivnost)

Funkcionalnost koju testiramo:

Ispitni slučaj:

Očekivani rezultati:

Postupak provođenja ispitivanja:

Dobiveni rezultati:

Status: PROLAZ

▸ [7. Tehnologije za implementaci...](#)

▸ [8. Upute za puštanje u pogon](#)

▸ [9. Zaključak i budući rad](#)

▸ [A. Popis literature](#)

▸ [A. Dnevnik promjena dokume...](#)

▸ [B. Prikaz aktivnosti grupe](#)

+ Add a custom sidebar

Clone this wiki locally

<https://github.com/KristijanOr>



```
{  
  "error": "Invalid token"  
}
```



Status: PROLAZ



Test 05: Funkcionalnost - Upravljanje korisnicima (Brisanje)

Funkcionalnost koju testiramo:

Administrativna metoda za brisanje korisničkog računa putem rute DELETE
/api/admin/users/:id

Ispitni slučaj 1: Uspješno brisanje

- Ulazni podaci: ID korisnika: 10 + admin token
- Rezultat: (200 OK)
- Tijelo odgovora:

```
{  
  "message": "User deleted"  
}
```



Status: PROLAZ

Ispitni slučaj 2: Korisnik ne postoji (idempotentnost)

- Ulaz: ID korisnika: 999 + admin token
- Rezultat: (200 OK)
- Tijelo odgovora:

```
{  
  "message": "User deleted"  
}
```



- Napomena: Sustav se ponaša idempotentno – potvrđuje brisanje čak i ako korisnik više ne postoji

Status: PROLAZ



Test 06: Funkcionalnost - Pogreške i Rutiranje

Funkcionalnost koju testiramo:

Ispitivanje stabilnosti sustava pri neočekivanim pogreškama.

Ispitni slučaj 1: Nepostojeća ruta

- Putanja: /api/nepostojece-putanje
- Rezultat: (404 Not Found)
- Tijelo odgovora:

```
{}
```



Status: PROLAZ

Ispitni slučaj 1: Kritični pad baze podataka

- Opis: Simulacija prekida veze s bazom (Database Exception)
- Rezultat: (500 Internal Server Error)
- Tijelo odgovora:

```
{
  "error": "Greška na serveru"
}
```



Status: PROLAZ

Ispitivanje sustava

Test 01: Registracija korisnika

Funkcionalnost koju testiramo:

Registracija novog korisnika u sustav.

Ispitni slučaj:

Ulazni podaci:

- Ime: Test
- Prezime: User
- Email: test.user@example.com
- Odabrana uloga: User

Očekivani rezultati:

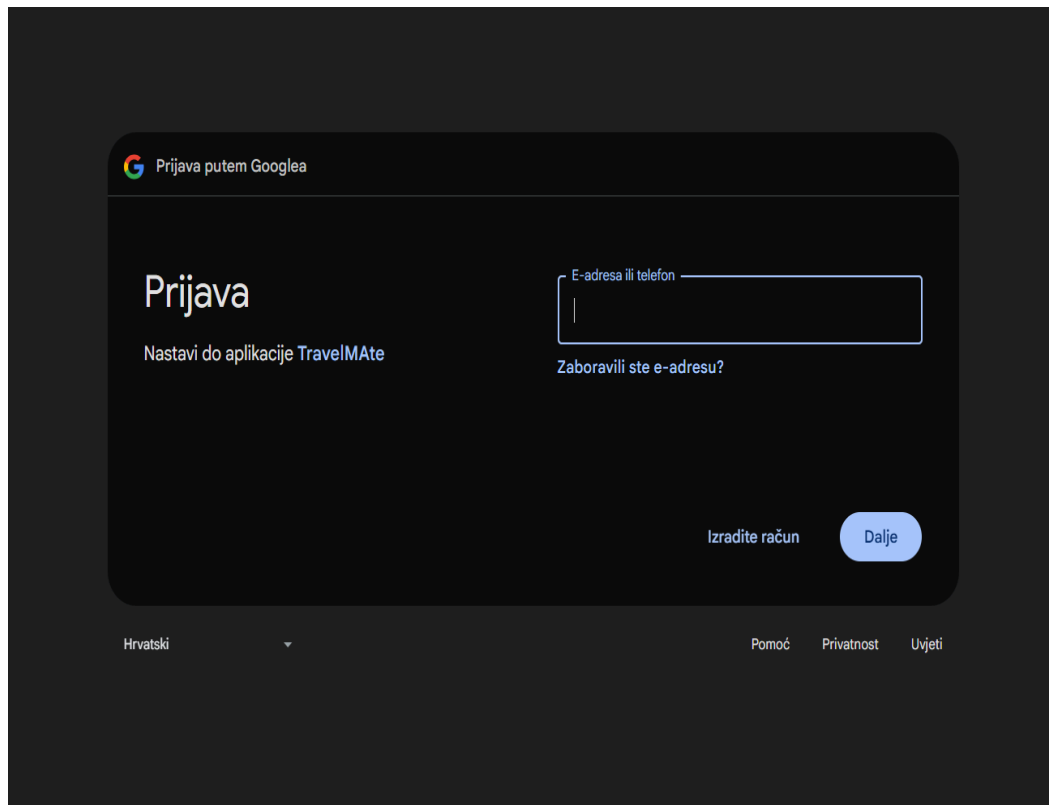
- Korisnik je preusmjeren na Google autentifikaciju.
- Sustav potvrđuje registraciju.

Postupak provođenja ispitivanja:

- Otvorena početna stranica aplikacije (<http://localhost:3000/>).
- Uneseni podaci u polja za ime, prezime i email.
- Odabran gumb za registraciju.

Dobiveni rezultati:

- Korisnik je preusmjeren na Google autentifikaciju i nakon toga registriran (Selenium ne može raditi u OAuth-u)
- Zbog toga se screenshot snimi na autentifikaciji.



Status:  PROLAZ

Test 02: Validacija – prazna forma

Funkcionalnost koju testiramo:

Provjera validacije prilikom slanja prazne registracijske forme.

Ispitni slučaj:

Ulazni podaci:

- Prazna forma (bez unosa podataka)

Očekivani rezultati:

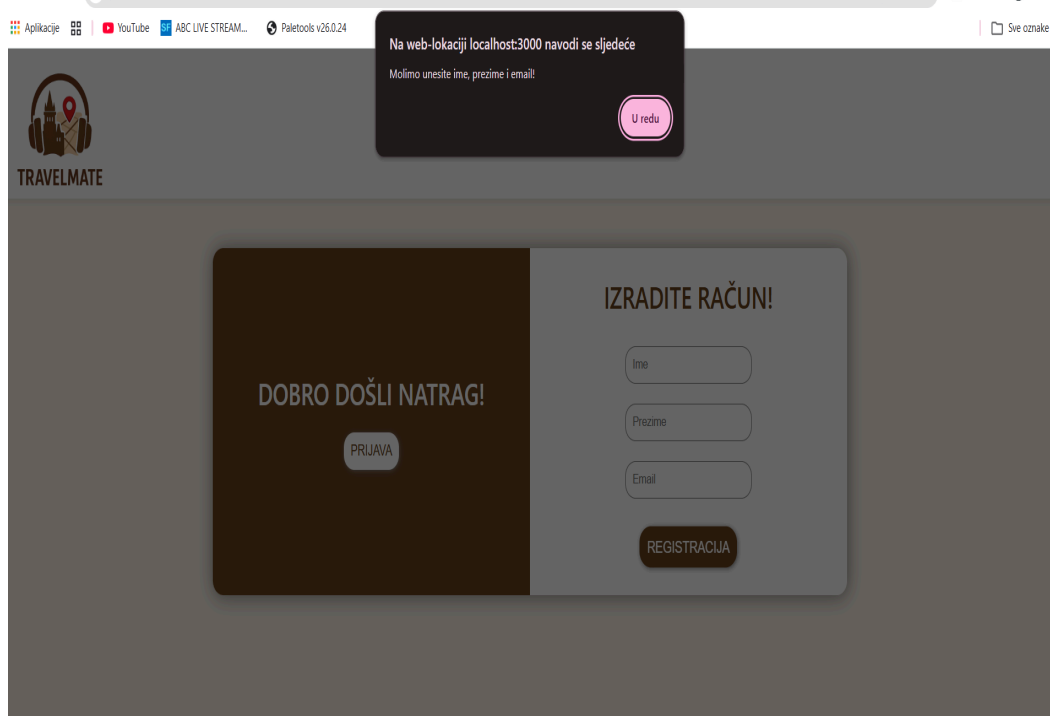
- Sustav prikazuje upozorenje (alert) o obaveznim poljima.

Postupak provođenja ispitivanja:

- Otvorena početna stranica aplikacije.
- Odabran gumb za registraciju bez unosa podataka.
- Čekanje pojave browser alerta.

Dobiveni rezultati:

- Alert je uspješno uhvaćen i validacija je ispravno provedena.



Status:  **PROLAZ**

Test 03: Zabrana pristupa admin funkcijama

Funkcionalnost koju testiramo:

Onemogućavanje pristupa administratorskim funkcijama neovlaštenim korisnicima.

Ispitni slučaj:

Ulazni podaci:

- Prijavljeni korisnik bez administratorskih ovlasti

Očekivani rezultati:

- Pristup administratorskoj stranici je zabranjen.

Postupak provođenja ispitivanja:

- Prijava korisnika bez admin ovlasti.
- **Odabran gumb za prijavu.
- Pokušaj pristupa administratorskoj ruti.

Dobiveni rezultati:

- Pristup je ispravno onemogućen.
- Na stranici se prikazuje poruka "Niste prijavljeni" što ukazuje na to da korisnik nema ovlasti za pristup toj stranici.

Izvođenje Test 3: Provjera zabrane pristupa na /admin...
čekam stabilizaciju UI-ja...
[15112:18648:0121/195900.785:ERROR:google_api\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[15112:18648:0121/195900.821:ERROR:google_api\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[15112:18648:0121/195900.821:ERROR:google_api\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[15112:18648:0121/195900.855:ERROR:google_api\gcm\engine\mcs_client.cc:700] Error code: 401 Error message: Authentication Failed: wrong_secret
[15112:18648:0121/195900.855:ERROR:google_api\gcm\engine\mcs_client.cc:702] Failed to log in to GCM, resetting connection.
✓ Sustav ispravno prepoznao neovlašten pristup.



TRAVELMATE

Niste prijavljeni

Status: ☒ PROLAZ

Test 04: Prikaz mape i favorita

Funkcionalnost koju testiramo:

Ispravan prikaz interaktivne mape atrakcija.

Ispitni slučaj:

Očekivani rezultati:

- Mapa se uspješno učitava i prikazuje.
- Prikaz svih favorita koji se toga trenutka nalaze u bazi podataka.

Postupak provođenja ispitivanja:

- Otvorena stranica s mapom. (/map)
- Provjereno postojanje map komponente.
- Provjereno postojanje favorita.

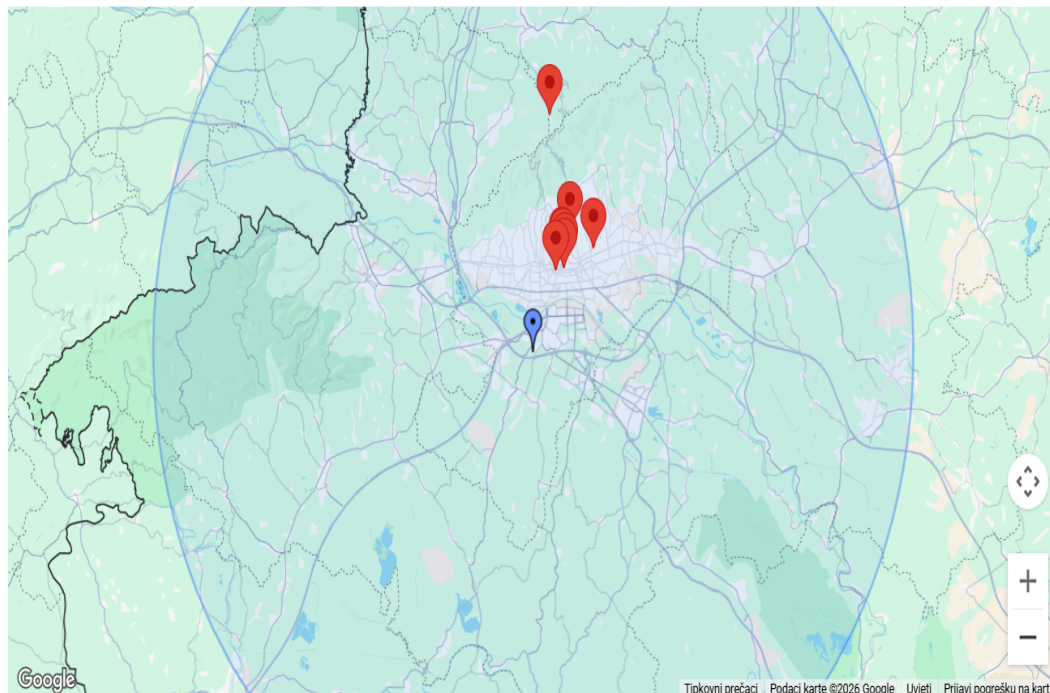
Dobiveni rezultati:

- Mapa i favoriti se uspješno prikazuju.



TRAVELMATE

[POVRATAK NA POČETNU STRANICU](#)



Status: ☒ **PROLAZ**

Test 05: Prikaz mobilne verzije (responzivnost)

Funkcionalnost koju testiramo:

Responzivnost aplikacije na mobilnim uređajima.

Ispitni slučaj:

Očekivani rezultati:

- Sučelje je pravilno prilagođeno mobilnom prikazu.

Postupak provođenja ispitivanja:

- Promjena veličine viewporta na mobilne dimenzije.
- Provjera prikaza elemenata.

Dobiveni rezultati:

- Mobilni prikaz je ispravan, aplikacija ima responzivan dizajn.

DOBRO DOŠLI NATRAG!

PRIJAVA

IZRADITE RAČUN!

Ime

Prezime

Email

REGISTRACIJA

Status:  PROLAZ

TravelMates



7. Tehnologije za implementaciju aplikacije

KristijanOresic edited this page 2 hours ago · [6 revisions](#)

Edit

New page

Programski jezici

JavaScript je korišten kao glavni programski jezik na klijentskoj i poslužiteljskoj strani aplikacije. Na klijentskoj strani koristi se u kombinaciji s React bibliotekom, dok se na poslužiteljskoj strani koristi unutar Node.js okruženja.

Radni okviri i biblioteke

Frontend:

- React (verzija 19.2.0)** : Korišten za razvoj dinamičkog, komponentnog korisničkog sučelja. Omogućava brzo i učinkovito upravljanje stanjem i renderiranje elemenata na stranici
- React Router (verzija 7.9.5)** : Koristi se za upravljanje navigacijom između različitih stranica u aplikaciji bez ponovnog učitavanja stranica
- Axios (verzija 1.13.1)** : Za slanje HTTP zahtjeva s frontend dijela aplikacije prema backend serveru
- Vite (verzija 7.3.1)** : Korišten kao alat za razvoj i izgradnju frontend aplikacije zbog bržeg pokretanja razvojnog okruženja i optimizirane produkcijske izgradnje.

Backend:

- Node.js (verzija 16+)** : Poslužiteljski okvir za pokretanje backend aplikacija
- Express (verzija 5.1.0)** : Backend framework za izradu RESTful API-ja koji omogućuje brzi razvoj servera i upravljanje HTTP zahtjevima
- Passport.js i JWT (verzija 9.0.2)** : Za autentifikaciju i autorizaciju korisnika putem tokena i Google OAuth 2.0
- bcrypt (verzija 6.0.0)** : Za sigurno šifriranje korisničkih lozinki
- pg (verzija 8.16.3)** : Za rad s PostgreSQL bazom podataka

Baza podataka

- PostgreSQL (verzija 13 ili novija)** : Korištena je kao relacijska baza podataka, database.sql projekt koristi SQL upite za interakciju s bazom

Razvojni alati

- Visual Studio Code** : Korišten kao IDE za razvoj aplikacije
- Git (verzija 2.34)** : Korišten za verzioniranje koda

▼ Pages13

Find a page or section...

▶ Home

▶ 1. Opis projektnog zadatka

▶ 2. Analiza zahtjeva

▶ 3. Specifikacija zahtjeva sustava

▶ 4. Arhitektura i dizajn sustava

▶ 5. Arhitektura komponenata i r...

▶ 6. Ispitivanje programskog rješ...

▼ 7. Tehnologije za implementaci...

Programski jezici

Radni okviri i biblioteke

Frontend:

Backend:

Baza podataka

Razvojni alati

Alati za ispitivanje

Alati za razmještaj

Cloud platforma

Obrazloženje izbora tehnologija

▶ 8. Upute za puštanje u pogon

▶ 9. Zaključak i budući rad

▶ A. Popis literature

▶ A. Dnevnik promjena dokume...

▶ B. Prikaz aktivnosti grupe

+ Add a custom sidebar

Clone this wiki locally

<https://github.com/KristijanOr>

Alati za ispitivanje

- **Vitest (verzija 4.0.17)** : Korišten za pisanje i izvođenje testova backend aplikacije. Omogućuje mockanje modula, simulaciju pogrešaka i provjeru očekivanih rezultata pomoću asercija (expect).
- **Supertest (verzija 7.2.2)** : Koristi se za testiranje HTTP ruta Express aplikacije slanjem simuliranih zahtjeva i provjerom statusnih kodova i odgovora servera.
- **Mockanje PostgreSQL baze** : Baza podataka (pg.Pool) mockana je pomoću Vitest alata kako bi se testirali scenariji poput nepostojećih ruta i grešaka na serveru bez korištenja stvarne baze.
- **Selenium WebDriver (verzija 4.40.0)** : Korišten za sustavno testiranje web aplikacije simulacijom stvarnog korisničkog ponašanja u pregledniku Google Chrome.

Alati za razmještaj

- **Docker (verzija 20.10)** : Korišten za kontenerizaciju aplikacije i omogućavanje lakše distribucije i pokretanja u različitim okruženjima

Cloud platforma

- **Render** : Korišten za implementaciju i hostanje aplikacije i PostgreSQL baze podataka u produkciji. Render je PaaS (Platform as a Service) rješenje koje omogućuje jednostavno upravljanje i skaliranje aplikacija bez potrebe za vlastitim infrastrukturnim postavkama

Obrazloženje izbora tehnologija

- React je odabran zbog svoje učinkovitosti u razvoju interaktivnih korisničkih sučelja s mogućnošću ponovne upotrebe komponenti i lakoće održavanja.
- Node.js i Express omogućuju brz i skalabilan razvoj API-ja, dok PostgreSQL nudi robusnu relacijsku bazu podataka koja je vrlo učinkovita za složenije upite i transakcije.
- Passport.js i JWT pružaju sigurnost i jednostavnost implementacije korisničke autentifikacije.
- Axios je odabran zbog svoje jednostavne upotrebe u slanju HTTP zahtjeva prema backendu.

TravelMates




```
SESSION_SECRET=mysecretkey
FRONTEND_URL=
GOOGLE_CALLBACK_URL=
NODE_ENV=
PORT=
```

- Postaviti drugu .env datoteku u frontend direktorij sa sljedećim sadržajem:

```
VITE_GOOGLE_MAPS_KEY=AIzaSyD8KE0YzHUf2UvPPRGCEr0fmJS-e70jGkk
VITE_SECRET_PASSWORD=travelmate2025
VITE_BACKEND_URL=
```

Postavke baze podataka: Upute za inicijalizaciju baze podataka, uključujući migracije i postavljanje inicijalnih podataka.

- Baza je globalno dostupna na Render javnom poslužitelju te ju dohvaćamo putem DATABASE_URL varijable u .env datoteci.
- Inicijalizacija baze :

```
cd backend
node node/initDB.js
```

3. Pokretanje aplikacije

Upute za pokretanje aplikacije:

- Pokretanje backenda:

```
node node/server.js
```

- Pokretanje frontenda:

Razvojno okruženje:

```
npm run dev
```

- Nakon uspješnog pokretanja, terminal će prikazati poruku s URL-om na kojem je aplikacija dostupna:

<http://localhost:5173/>

Produkcijsko okruženje:

```
npm run build
```

Pokretanje poslužitelja:

```
npm install -g serve
serve -s dist
```

4. Upute za administratore

Smjernice za administratore aplikacije nakon puštanja u pogon:

▸ [9. Zaključak i budući rad](#)

▸ [A. Popis literature](#)

▸ [A. Dnevnik promjena dokume...](#)

▸ [B. Prikaz aktivnosti grupe](#)



+ Add a custom sidebar

Clone this wiki locally

<https://github.com/KristijanOr>



- **Pristup administratorskom sučelju:**

- URL za admin panel : /secret-admin-register.
- Početni podaci za prijavu :

Admin lozinka : travelmate2025



Redovito održavanje:

- Ažuriranje aplikacije (primjer: povlačenje novih verzija iz Git repozitorija i ponovno pokretanje aplikacije).
- Ponavljanje naredbi s početka ovog poglavlja

5. Postavljanje na Render:

- Prijavite se na [Render](#).
- Kreirajte novi **Web Service** i povežite ga s vašim GitHub repozitorijem.
- Dodajte environment varijable (npr. DATABASE_URL, API_KEY).
- **Pokretanje aplikacije:**

Render će automatski preuzeti repozitorij, instalirati ovisnosti i pokrenuti aplikaciju.

Nakon deploja, aplikaciji možete pristupiti putem generiranog URL-a (npr. <https://my-web-app.onrender.com>).

Opis pristupa aplikaciji na javnom poslužitelju

Pristup aplikaciji

Pristup za user role

- Uđite na stranicu : <https://travelmates-2.onrender.com>
- Pričekajte da Render pokrene stranicu
- Registrirajte se, otvorite kartu i učitati će se znamenitosti na interaktivnoj karti
- Renderiranje karte traje do 5 sekundi
- Ako odobrite lokaciju, vidite sve znamenitosti iz baze u radijusu od 40 km oko vas (8 njih)
- Ako ne odobrite lokaciju, vidite sve znamenitosti iz baze, a lokacija vam je postavljena na središte Hrvatske
- Klikom na marker znamenitosti vidite sliku, naziv, opis, radno vrijeme, udaljenost od vaše lokacije i dvije ikone
- Ako odaberete ikonu srca, ta znamenitost će postati vaš favorit
- Ako odaberete ikonu zvučnika, čuti ćete opis znamenitosti pomoću TTS-a (Text-To-Speech)
- Ponovnim klikom na ijednu od ikona, ili mičete znamenitost iz favorita ili pauzirate TTS
- U gornjem desnom kutu /map stranice se također nalazi gumb s ikonom srca koji vas kada ga pritisnete preusmjeri na stranicu s vašim favoritima
- Na toj stranici vidite sve vaše favorite koje možete obrisati
- Također se na /user stranici uz opciju otvaranja karte, vide vaši favoriti koje također možete obrisati ako želite

Pristup za admin i editor role

- Uđite na stranicu : <https://travelmates-2.onrender.com/secret-admin-register>
- Tražit će vas admin lozinku koja je "travelmate2025"
- Nakon uspješno unesene lozinke, moći ćete se registrirati i kao editor i kao admin i kao user
- Admin vidi sve korisnike aplikacije, bili to useri, editori ili admini te im može promijeniti role ili obrisati ih
- Editor vidi sve atrakcije, može dodati nove atrakcije i obrisati ili urediti već postojeće

Ograničenje

- Na Renderu su deployani backend i frontend odvojeno, te zbog toga imamo poseban URL za backend i URL za frontend
- Zbog toga imamo ograničenje da ova web aplikacija radi u Google-u i u Microsoft Edge-u, ali ne radi u Firefox-u i u bilo kojem mobilnom pregledniku
- To se događa jer ti preglednici ne dopuštaju slanje cookie-a kada aplikacije imaju različite URL-ove za backend i frontend, smatraju da su to Third-Party Cookies

TravelMates



9. Zaključak i budući rad

vitapavlovic edited this page 1 hour ago · 3 revisions

Zaključni pregled izrade projektnog zadatka

Vrijeme izrade

Razvoj aplikacije TravelMate predstavljao je složen i zahtjevan proces koji je tražio dobru organizaciju rada, stalnu suradnju članova tima i prilagodbu različitim tehničkim i vremenskim ograničenjima. Realizacija projekta trajala je cijeli semestar te je zahtijevala puno uložениh sati rada. Cjelokupan trud uložen u izradu aplikacije doprinio je boljem razumijevanju razvoja složenijih web sustava i timskog rada u realnim uvjetima te je doveo do razvoja stabilne i funkcionalne verzije aplikacije.

Tehnički izazovi

Tijekom izrade projekta suočili smo se s nekoliko tehničkih izazova. Jedan od njih bila je integracija frontend dijela aplikacije s backend sustavom, što je zahtijevalo pravilno definiranje API poziva i usklađivanje formata podataka između klijenta i poslužitelja. Također, implementacija prijave korisnika putem OAuth 2.0 sustava bila je zahtjevna zbog potrebe za razumijevanjem sigurnosnih protokola, upravljanja autentifikacijskim tokenima i kontrole pristupa prema korisničkim ulogama. Dodatni izazov predstavljao je proces postavljanja aplikacije u produkcijsko okruženje na Render platformi, koji je zahtijevao prilagodbu konfiguracija i osiguravanje ispravne komunikacije između frontend i backend dijela aplikacije. Osim toga, integracija vanjskih servisa poput Google Maps API-ja i Text-to-Speech sustava zahtijevala je pažljivo upravljanje API ključevima i osiguravanje stabilnog rada funkcionalnosti.

Stečena znanja

Rad na projektu omogućio je članovima tima stjecanje i unapređenje brojnih tehničkih i organizacijskih vještina. Tehnička znanja s kojima se sada znamo bolje koristiti uključuju React, Vite, Node.js, Express, PostgreSQL bazi te integraciji vanjskih sustava poput Text-to-Speech i Google Maps. Osim toga, projekt je doprinio razvoju vještina u koordinaciji, planiraju i komunikaciji što je poboljšalo efikasnost i uspješnost zajedničkog rada te imalo pozitivan utjecaj na kvalitetu konačnog rješenja.

Potrebna posebna znanja

Za brže i kvalitetnije ostvarenje projekta korisno bi bilo unaprijed posjedovati naprednija znanja iz područja frontend i backend razvoja, rada s bazama podataka te autentifikacije i autorizacije korisnika. Također, veće iskustvo u korištenju alata za upravljanje projektima poput GitHuba, dodatno bi unaprijedilo organizaciju rada i praćenje napretka unutar tima.

Pages13

Find a page or section...

Home

1. Opis projektnog zadatka

2. Analiza zahtjeva

3. Specifikacija zahtjeva sustava

4. Arhitektura i dizajn sustava

5. Arhitektura komponenata i r...

6. Ispitivanje programskog rješ...

7. Tehnologije za implementaci...

8. Upute za puštanje u pogon

9. Zaključak i budući rad

Zaključni pregled izrade projektnog zadatka

Vrijeme izrade

Tehnički izazovi

Stečena znanja

Potrebna posebna znanja

Perspektive za nastavak rada i funkcionalnosti

Zaključak

A. Popis literature

A. Dnevnik promjena dokume...

B. Prikaz aktivnosti grupe

+ Add a custom sidebar

Clone this wiki locally

https://github.com/KristijanOr

Perspektive za nastavak rada i funkcionalnosti

Aplikacija TravelMate ima značajan potencijal za daljnji razvoj i dodatna proširenja funkcionalnosti. U budućnosti bi se mogli implementirati napredni filteri i preporuke za znamenitosti na temelju interesa korisnika, kao i sustav ocjenjivanja i komentiranja atrakcija. U ostvarenoj verziji aplikacije TravelMate implementirane su sve ključne funkcionalnosti definirane funkcionalnim zahtjevima projekta. Jedina funkcionalnost koja nije implementirana je nefunkcionalni zahtjev za podršku više jezika (npr. engleski i hrvatski), koji je u fazi planiranja bio definiran kao zahtjev niskog prioriteta. Ta bi se funkcionalnost mogla implementirati kao potencijalna nadogradnja u budućim verzijama aplikacije, čime bi se dodatno unaprijedila aplikacija, poboljšalo iskustvo korisnika te se povećala njezina vrijednost i dostupnost.

Zaključak

Projekt TravelMate uspješno je realiziran te predstavlja kvalitetno i funkcionalno rješenje, odgovara prethodno definiranim ciljevima i potrebama korisnika. Unatoč određenim izazovima, tim je razvio aplikaciju koju je moguće nastavljati nadograđivati i razvijati. Stečena znanja i iskustva čine dobru osnovu za budući rad na ovom i sličnim projektima.

A. Dnevnik promjena dokumentacije

Edit

New page

KristijanOresic edited this page yesterday · [5 revisions](#)

Rev.	Opis promjene/dodatka	Autori	Datum
0.1	Napravljen predložak	Kristijan Orešić	8.10.2025
0.2	Napravljen detaljan opis projektnog zadatka	Kristijan Orešić, Franko Pavlić	8.10.2025.
0.3	Postavljeni funkcionalni i nefunkcionalni zahtjevi	Kristijan Orešić, Franko Pavlić	17.10.2025.
0.4	Ažuriranje i poboljšanje funkcijskih zahtjeva	Franko Pavlić	27.10.2025.
0.5	Napravljeni dijagrami obrazaca uporabe	Nola Bonačić Dorić, Vita Pavlović	28.10.2025.
0.6	Napravljen opis i odabir arhitekture sustava	Kristijan Orešić	11.11.2025.
0.7	Detaljno opisana organizacija aplikacije	Franko Pavlić	13.11.2025.
0.7	Ispunjene tablice baze podataka, napravljen dijagram baze podataka	Kristijan Orešić	13.11.2025.
0.8	Napravljen dijagram razreda za bazu podataka	Nik Čolić	13.11.2025.
0.9	Ažuriran Dijagram obrazaca uporabe za funkcionalnost administratora	Nola Bonačić-Dorić	13.11.2025.
0.10	Napravljen opis baze podataka	Kristijan Orešić	14.11.2025.
0.11	Napravljeni nedostajući dijagrami obrazaca uporabe	Nola Bonačić-Dorić	14.11.2025.
0.12	Dovršavanje dokumentacije za prvu predaju	Svi članovi	14.11.2025.
0.13	Napravljeni dijagrami komponenata i razmještaja	Nola Bonačić-Dorić	19.01.2025.
0.14	Napravljeni dijagrami stanja i aktivnosti	Nik Čolić	21.01.2025.
0.15	Napravljena ispitivanja programskog rješenja	Kristijan Orešić, Vita Pavlović, Ognjen Škarić	21.01.2025.
0.16	Dovršavanje dokumentacije za drugu predaju	Svi članovi	23.01.2025.

▼ Pages13

Find a page or section...

▶ Home

▶ 1. Opis projektnog zadatka

▶ 2. Analiza zahtjeva

▶ 3. Specifikacija zahtjeva sustava

▶ 4. Arhitektura i dizajn sustava

▶ 5. Arhitektura komponenata i r...

▶ 6. Ispitivanje programskog rješ...

▶ 7. Tehnologije za implementaci...

▶ 8. Upute za puštanje u pogon

▶ 9. Zaključak i budući rad

▶ A. Popis literature

A. Dnevnik promjena dokume...

▶ B. Prikaz aktivnosti grupe

+ Add a custom sidebar

Clone this wiki locally

https://github.com/KristijanOr

- Postavljanje rokova do kojih određene funkcionalnosti moraju biti donekle napravljene

6. sastanak

- Datum: u ovom formatu: 12. listopada 2025.
- Prisustvovali: K.Orešić, F.Pavlić, N.Bonačić Dorić, V.Pavlović, N.Čolić, O.Škarić
- Teme sastanka:
 - Prikaz dosad implementiranih funkcionalnosti (Login + Google Maps API)
 - Dogovor oko raspodjele posla za dovršavanje ostatka dokumentacije pred prvu predaju

7. sastanak

- Datum: u ovom formatu: 7. siječnja 2026.
- Prisustvovali: K.Orešić, F.Pavlić, N.Bonačić Dorić, V.Pavlović, N.Čolić, O.Škarić
- Teme sastanka:
 - Dogovor oko raspodjele posla na web aplikaciji za završnu verziju
 - Dogovor oko raspodjele posla na dokumentaciji za drugu predaju

8. sastanak

- Datum: u ovom formatu: 14. siječnja 2026.
- Prisustvovali: K.Orešić, F.Pavlić, N.Bonačić Dorić, V.Pavlović, N.Čolić, O.Škarić
- Teme sastanka:
 - Prikaz alfa verzije web aplikacije (dobivene povratne informacije)
 - Diskusija o mogućim rješenjima za završne probleme oko aplikacije

8. sastanak

- Datum: u ovom formatu: 21. siječnja 2026.
- Prisustvovali: K.Orešić, F.Pavlić, N.Bonačić Dorić, V.Pavlović, N.Čolić, O.Škarić
- Teme sastanka:
 - Završena web aplikacija
 - Deploy aplikacije na Render i transfer baze s Dockera na Render

Tablica aktivnosti

U tablici aktivnosti nalazi se procijenjen broj sati koji su članovi projektnog tima proveli u izgradnji određenih dijelova sustava.

	Kristijan Orešić	Franko Pavlić	Nik Čolić	Nola Bonačić-Dorić	Vita Pavlović	Ognjen Škarić
Upravljanje projektom	20					
Opis projektnog zadatka	2					
Funkcionalni zahtjevi		2				
Opis pojedinih obrazaca			2	5	5	

	Kristijan Orešić	Franco Pavlić	Nik Čolić	Nola Bonačić-Dorić	Vita Pavlović	Ognjen Škarić
Dijagram obrazaca				5	5	
Sekvencijski dijagrami				3	5	
Opis ostalih zahtjeva		3				
Arhitektura i dizajn sustava	2	5				
Opis baza podataka	3		10			
Dijagram razreda			3			
Dijagram stanja			5			
Dijagram aktivnosti				5		
Dijagram komponenti				5		
Korištene tehnologije i alati		3				
Ispitivanje programskog rješenja	10				10	10
Dijagram razmještaja				5		
Upute za puštanje u pogon		2				
Dnevnik sastajanja	2					
Zaključak i budući rad	2					
Implementacija baze podataka	5		15			
Implementacija poslužiteljske strane	60	45	10	5	5	20
Dizajniranje korisničkog sučelja	5	10		30	30	
Implementacija klijentske strane	30	30		5	5	10
Puštanje u pogon	20					

Dijagram pregleda promjena

Standardni GitHubov grafikon doprinosa (Contributors) na ovom repozitoriju može djelovati nepotpuno jer prikazuje statistiku isključivo za main granu. Kako smo koristili Feature Branch Workflow (svaki član tima radio je u vlastitoj grani radi sigurnosti koda), većina rada svih članova vidljiva je kroz Merge operacije, a ne kroz izravne commitove u main. Za stvarni uvid u aktivnost, potrebno je pogledati povijest svih grana.

